Trading Mantissa Bits for Watts: Software Mantissa Zeroing under Module Power Caps on the NVIDIA GH200

Koying Chen
Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan

***Abstract*—We study a software knob exploiting data-dependent GPU power—zeroing unneeded low-order mantissa bits of FP64, BF16 and FP8 operands—on 14 NVIDIA GH200 nodes under module power caps, with paired measurements and an equivalent-power metric (the extra cap the unmodified program needs). Under a binding cap, every tested GEMM-dominated HPC program that truncates internally and checks its results gains. At 500–800 W, tuned 8-node HPL keeping 36 significand bits (35 fraction bits; $\approx 100\times$ residual margin) runs 3.3–4.7% faster over 6 pairs on two node sets, worth 42–74 W per node at 500–700 W, and HPL-MxP keeping 3 bits 1.7–5.8% on a diagonally dominant test matrix; at 450–550 W Quantum ESPRESSO, with bits tied to its Davidson tolerance, runs 2.1–3.6% faster (partly from fewer iterations) with all printed energy digits unchanged; at 500–700 W DF-MP2 keeping 20–32 bits runs 4.7–7.7% faster, and a wide-range Gaussian-process likelihood keeping 28 bits 7.5–7.9%, where FP32 fails and INT8 emulation is slower. At the default 1000 W cap HPL uses 7.5–8.6% less energy per solve and is 0.6–3.9% faster, depending on how far the cap and temperature lower the native clock. In 2–4 runs per setting at 1000 W, fill controls attribute 62–97% of the saving to bits that stop toggling. Safe bit counts need validation across problem sizes; the crossover with INT8 emulation of FP64 moves with matrix shape and cap. With wide per-row exponent spans, fixed-width INT8 emulation can silently fail: at 36 bits it shifts a Gaussian-basis DFT energy by 52.5 $\mathrm{mE_h}$, whereas rounding to 36 bits errs by 10^{-6} $\mathrm{E_h}$. All-FP32 or 3xTF32 GEMMs fail on long inner products, and in Hopper AI inference FP8 already captures most of the benefit.**

***Index Terms*—power capping, energy efficiency, floating-point precision, mantissa truncation, GPU, mixed precision, HPL**

I. Introduction

Power is becoming the ceiling on delivered performance. Data centers cap rack power, student cluster competitions cap the power of a whole cluster, and superchips such as the NVIDIA GH200 let users cap the power of the entire module. Under a cap, the power manager lowers the clock until the module fits; the speed of a program then depends on how much energy each of its operations consumes.

That GPU power depends on input data is known. Bhalachandra et al. measured 67% more power for DGEMM on random than on constant operands on A100 [1]; Gregersen et al. varied the distribution and bit patterns of GEMM inputs and found that zeroing the least significant bits lowers power [2]; He showed an all-zero matrix multiplication on A100 running about 15% faster than a random one, with a gap that widens as the power limit is lowered [3]; Antepara et al. measured 563 W for a zero-input DGEMM on GH200 versus 898 W for random input [4]; and the TensorRT documentation warns that all-zero inputs draw less power because fewer DRAM and L2 bits toggle [5]. In hardware, Tong et al. narrowed floating-point multipliers [6], Pool et al. selected precision for graphics shaders [7], [8], Guenther et al. masked mantissas at the inputs of arithmetic units [9], and an AMD patent adds a hardware zeroing module to shaders [10].

These works either demonstrate the effect with special inputs such as all-zero or constant data (He also swept the power limit, but only for all-zero against random inputs) or require new hardware or simulated energy. None answers three practical questions. How much performance does software truncation of the low-order mantissa bits of *real* data buy under a power cap on a production GPU? How much accuracy does it cost? And where does it still pay off against format changes such as FP8 or INT8 emulation of FP64 [11], [12]? To our knowledge this is the first study that truncates part of the mantissa of real FP64/BF16 operands in software on a production GPU and measures, on multi-node benchmarks and real HPC applications checked by their own correctness criteria, how much throughput the saved switching power buys under power caps. Our contributions are:

1) **Gains on real applications under power caps.** We express the benefit as the extra cap the unmodified program needs to catch up, from GEMM to tuned 8-node HPL (36 significand bits: 3.3–4.7% faster at 500–800 W, 42–74 W at 500–700 W, 6 pairs on two node sets), HPL-MxP on a diagonally dominant test matrix, Quantum ESPRESSO, power-hungry DF-MP2 (4.7–7.7% at 500–700 W) and a wide-range, power-bound Gaussian-process likelihood (7.5–7.9% at 500–700 W), all with paired 95% intervals, and check residuals and energies with each code's own criterion (Sections VI-A–VI-E, VII-E).
2) **The mechanism, across compute and data movement.** Filling the discarded bits with ones or a fixed pattern instead of zeros separates "no toggling" from "value is zero" in tensor-core GEMM, FP64 FMA units and memory traffic, quantifying the toggling hypothesis on GH200 module power (Section III).
3) **Tolerance-driven bit counts and failure boundaries.** A rule that uses only the solver's own tolerance leaves every printed digit of the Quantum ESPRESSO energies unchanged; we measure how far HPL (several

sizes), PySCF, third-party non-GEMM codes, LoRA fine-tuning and six language models can be truncated, and the role of the rounding mode (Sections VI–VIII).

4) **The boundary with format changes.** Against INT8 emulation (cuBLAS, GEMMul8) and FP32/3xTF32 for FP64, and FP8 for AI, the crossover moves with matrix shape, power cap, accumulation precision and exponent span; on data with a wide per-row exponent span, fixed-width INT8 emulation can silently return wrong results, a risk that a cheap measurement of per-row spans flags beforehand (Sections VII–VIII).

Compared with the closest work: Gregersen et al. observed input-dependent power and proposed the toggling hypothesis but avoided throttling and studied only GEMM; He swept the A100 power limit, and Antepara et al. measured GH200, but both contrasted all-zero with random inputs without truncating real data or evaluating accuracy. We quantify the toggling hypothesis on GH200 module power, show that the data-movement part is consistent with DRAM energy studies [13], [14], and quantify speed and accuracy on real applications under caps. Uchino et al. evaluated the speed and energy efficiency of INT8-emulated GEMM [12]; we compare it with truncation at the same cap and comparable accuracy and find that the winner depends on matrix shape and cap.

II. Background and Related Work

Input-dependent GPU power. Bhalachandra et al. argue that power budgets and benchmarks must account for input entropy [1]. Gregersen et al. varied value distribution, bit similarity, ordering and sparsity of CUTLASS GEMM inputs on A100, H100 and V100 and saw power differ by up to almost 40%, with zeroed low bits lowering power [2]; they chose sizes that do not throttle, so performance under a cap was not measured. He reported 295 versus 257 TFLOP/s for all-zero versus random 8192^3 GEMMs on A100 and, lowering the power limit from 330 to 100 W, saw the relative speedup of such predictable inputs grow from about 1.2× to about 2.3× at 150 W before it fell at 100 W [3]. Antepara et al. measured zero-input DGEMM on GH200 at 563 W and 65.4 TFLOP/s against 898 W and 57.8 TFLOP/s for random input [4]; their GH200 roofline study found no significant difference in vector FP64 performance between zero and random inputs [15], consistent with our finding that when the GPU is not power-bound the difference shows only in power. Ujeniya et al. note that constant and random data can differ by about 100 W on H100/H200 but leave it out of scope [16]. Lucas and Juurlink built a data-dependent ALU power model for GTX 580 whose main term is the number of toggles, with the number of ones a secondary term [17]. All of these use all-zero, constant or special bit patterns; none truncates part of the mantissa of real data or evaluates accuracy or applications.

Power-constrained HPC. Rountree et al. took a first look at performance under a hardware-enforced power bound and found that manufacturing variation makes nominally identical processors differ by about 6% in execution time under the same bound [18]; Inadomi et al. measured up to 64% variation in per-rank DGEMM performance under a module power cap on a large system and allocated power by variability [19]. Patki et al. overprovision nodes under a fixed system power budget and enforce per-node bounds [20], and Zeus tunes the GPU power limit together with the batch size to cut DNN training energy [21]. These works decide how power is distributed; we lower the energy each operation needs, so the two compose. The 13–17% node-to-node clock spread we see at equal caps (Section V) is consistent with their observations.

Hardware mantissa masking and data-movement energy. Mantissa masking itself is not new. Tong et al. showed that many programs tolerate halving the mantissa and that a variable-width multiplier saves up to 66% energy per operation [6]. Pool et al. estimated 23–36% energy savings in the vertex stage of mobile GPUs [7] and up to 71% in pixel-shader arithmetic (10–20% of GPU energy) [8]; Angerd et al. automated precision reduction in graphics and designed a register file that packs arbitrary precisions [22]; Guenther et al. masked mantissas at functional-unit inputs of an LTE baseband processor for up to 23% energy per task [9]; an AMD patent zeroes mantissa bits in GPU shaders according to a precision flag [10]; and Jahadi and Atoofian exploited the zero bias of register bits and dropped low-order bits of CNN values to cut tensor-core register-file leakage by 77.3% in simulation [23]. rpe and chop use the same bit operations to emulate low precision but do not measure power [24], [25]. DRAM and HBM energy depend on data toggling rates [13], [14], [26], and compression can reduce bytes but increase toggles [27]. We differ in being pure software, changing neither hardware nor data format, and in measuring clocks and performance under caps on a production GPU with validated application accuracy.

Precision tuning, mixed precision and power caps. Tools such as Precimonious lower variable precision for speed [28], and mixed-precision storage reduces bytes per value [29]; mantissa zeroing changes neither type nor bytes and composes with both. Transprecision computing argues for controlling precision at fine grain in space and time [30]. Closest in form, Grützmacher et al. store FP64 values as mantissa segments that keep the full exponent and read only the leading segments, speeding up memory-bound PageRank by 10–30% [31]; there the truncation saves bandwidth, here switching power. Low-precision LU with iterative refinement recovers FP64 accuracy [32] and underlies HPL-MxP [33]. Klöwer et al. estimate from the data itself how many mantissa bits carry real information and round the rest to nearest to compress atmospheric data [34]; the same analysis could estimate in advance how many bits GEMM operands need, here for power rather than storage. GH200 shares module power between CPU and GPU and lowers the GPU clock near the cap [35]; Yang et al. found that nvidia-smi samples only about 25% of runtime on A100/H100 [36], which we mitigate with long steady windows, pairing and energy integrated from 1-s averaged power (Section V).

Emulating FP64 on low-precision units. The Ozaki scheme [37] splits a matrix product into a sum of products of

lower-precision slices that are computed exactly; Ootomo et al. run it on integer matrix units [38], and Ozaki scheme II uses modular arithmetic [11] and is implemented in GEMMul8, with performance and power evaluated by Uchino et al. [12]. cuBLAS ships an Ozaki-type fixed-point emulation that scales each row of A and column of B by a shared factor, so the dynamic range within a row is encoded in the mantissa; its automatic mode picks the slice count from the exponent span and dispatches to native FP64 when more than a library-defined maximum of bits would be needed [39], [40]. These change the arithmetic format; zeroing keeps the original FP64 (or BF16, FP8) units and lowers input entropy, so the two can be complementary.

Mixed precision in geostatistics. ExaGeoStat evaluates Gaussian-process likelihoods with an FP64 tile Cholesky [41]. Its mixed-precision successors order the locations along a space-filling curve so that strong correlations sit near the diagonal, keep diagonal tiles in FP64 and convert tiles chosen by their distance from the diagonal or their norm to FP32, FP16 or FP8: 1.6× on average with FP64/FP32 [42], up to 2.64× with FP64/FP32/FP16 [43], up to 12× when combined with tile low-rank compression [44], and 3× over FP64 on a GH200 with four precisions [45]; the same tile method drove a climate emulator on GH200 systems [46]. Cao et al. also measured the power and energy of such factorizations on single V100, A100 and H100 GPUs [47], without caps or a clock analysis. Uniform single precision is insufficient for these likelihoods [43], as we also find. All of these convert whole tiles to another format; we keep FP64 and truncate operand mantissas, and the two compose (Section VII-E). Olivares-Amaya et al. similarly ran RI-MP2 with single- and mixed-precision GEMMs on GPUs [48].

Convergence-driven precision. Woo et al. switch LOBPCG between FP32, FP64 and mixed precision by convergence state for up to 8.5× speedup on GPUs [49]; Huang, Shao and Hammond choose the number of INT8 emulation slices in GPU4PySCF from the relative SCF energy change, keeping errors below 10^{-7} hartree (E_h; $1\,E_h = 2\,Ry = 627.5$ kcal/mol) [50]; Dawson et al. found with MPFR that about 37 significant bits suffice for density matrix purification [51]. All of these gain speed by moving to a higher-throughput format or unit, and none studies power. We stay on FP64 units, adjust the kept bits one at a time from the Davidson threshold of a plane-wave code, and gain mainly from clock headroom under a cap (on one of our two systems also from fewer Davidson iterations; Section VI-D).

FP8 inference and its failures. FP8 training keeps high-precision master weights with per-tensor scaling, E4M3 for weights and activations and E5M2 for gradients [52]; Gupta et al. showed that rounding decides the fate of low-precision training [53]. The clearest FP8 failure is attention: FlashAttention-3's FP8 mode drops the VQA-a video-quality score of CogVideoX from 70.2 to 6.5 and of HunyuanVideo from 82.5 to 4.4, and 262k-context retrieval from 7.0 to 0.4, which SageAttention2 repairs inside FP8 by smoothing outliers and using a fixed scale for softmax probabilities [54]. MLPerf Inference closed division allows any pure, reproducible mathematical quantization but forbids pruning non-zero weights [55]; mantissa zeroing is the former.

III. Why Fewer Mantissa Bits Save Power

CMOS dynamic power is roughly $\alpha_{sw} C V^2 f$, where the switching activity α_{sw} depends on the data. Under a cap, the power manager lowers f (and V) until the module fits, so a lower α_{sw} turns directly into higher clock and throughput; below the cap it appears as lower power at the same clock. We see both on GH200. At the default 1000 W cap the native FP64 GEMM is still capped 95% of the time and runs at 1822 MHz, whereas keeping 0 bits it runs at the maximum 1980 MHz while module power falls from 984 to 707 W. Under a 700 W cap the FP64 GEMM SM clock is 1502 MHz native, 1668 MHz with 27 kept bits and 1965 MHz with 0 kept bits, all at about 694 W.

Toggling or zero? Zeroing changes two things at once: the low bits stop varying from element to element, and their value becomes zero. To separate the two we fill the discarded bits with all ones, or with a fixed nonzero pattern shared by all elements; neither toggles, but neither is zero. At 1000 W the fills with 0 kept bits run at the maximum 1980 MHz (with 20 kept bits at 1963–1971 MHz, partly capped), so we compare module power; since the cap holds the native GEMM at 984 W, the share we attribute to "no toggling" is conservative. At 500 W every configuration is capped 100% of the time, so we compare throughput (one node, two runs each; Table I).

In these measurements (2–4 runs per setting at 1000 W; one node and two runs at 500 W), merely stopping the low bits from toggling captures 68–92% of the benefit in tensor-core GEMM, quantitative support for the toggling hypothesis [2] on GH200 module power; with so few samples these shares are estimates, not an exact decomposition. Zeros save somewhat more than ones or a fixed pattern. With 20 kept bits this matches multiplier structure: partial products are $a_i b_j$, so low bits of a fixed at one still form partial products that toggle with the kept bits of the other operand, whereas zeros keep that whole slice at zero. With 0 kept bits both mantissas are constant, so the partial products do not toggle under any fill, yet zeros still save more than ones (707 versus 774 W); this may come from simpler alignment and accumulation of products that are exact powers of two, which we have not verified. We have no gate-level measurement, so both are inferences consistent with the data. The result also matches the GTX 580 ALU model of Lucas and Juurlink, in which toggles dominate and the number of ones in constant inputs is secondary [17].

Not just tensor cores. In an FP64 CUDA-core multiply-add loop (8×8 outer product per thread, no tensor cores), keeping 0 bits lowers module power at 1000 W from 862 to 553 W; ones and the fixed pattern keep 87% and 62% of that saving. In pure memory traffic (a STREAM-like copy at 3.33 TB/s) power falls from 672 to 549 W (−18%), with ones and the pattern keeping 97% and 86%. Fewer toggles on data wires and in arithmetic units are therefore the main cause, and the effect also exists for

TABLE I

FILLING THE DISCARDED BITS WITH ZEROS, ONES OR A FIXED PATTERN (TC: TENSOR CORE). 1000 W ROWS GIVE MODULE POWER IN W (2–4 RUNS EACH), 500 W ROWS THROUGHPUT IN TFLOP/S (ONE NODE, TWO RUNS EACH). THE LAST COLUMN IS THE SHARE OF THE ZERO-FILL BENEFIT THAT ONES AND THE FIXED PATTERN RETAIN.

Kernel, kept bits, cap	Native	Pattern	Ones	Zeros	Share
FP64 TC GEMM, 0, 1000 W	984	796	774	707	68–76%
FP64 TC GEMM, 0, 500 W	38.5	47.6	48.3	50.3	77–83%
FP64 TC GEMM, 20, 500 W	38.5	43.1	43.4	44.1	83–87%
BF16 TC GEMM, 0, 500 W	430	542	541	551	92%
FP64 FMA loop, 0, 1000 W	862	671	593	553	62–87%
Memory copy, 0, 1000 W	672	566	553	549	86–97%

bandwidth-bound code; the data-movement part is consistent with DRAM energy that depends on toggling rates [13], [14].

IV. METHOD

Mantissa zeroing. For a floating-point operand with p fraction bits (52 for FP64, 7 for BF16, 3 for FP8 E4M3) we keep the top k fraction bits and set the remaining $p-k$ to zero. Sign and exponent are untouched, so the representable range is the format's full range. We compare truncation toward zero (one bit mask), round-to-nearest (add half a unit of the last kept bit, then mask) and stochastic rounding (add a uniform random integer below the weight of the discarded bits, then mask). Ties go away from zero in the shim, the GEMM microbenchmark, Quantum ESPRESSO, DF-MP2 and the Gaussian-process code, and to even in HPL, the LoRA weight loading and LLM inference. We write "truncate to k bits" for keeping k bits either way: unless stated otherwise, the applications (Quantum ESPRESSO, PySCF, DF-MP2, Gaussian process, ExaGeoStat, third-party codes) and HPL round to nearest, while the GEMM microbenchmarks, the crossover sweeps and HPL-MxP truncate toward zero. HPL counts significand bits including the hidden bit, as rocHPL does: its 36 bits are 35 fraction bits. Each element costs one integer AND (plus an add for rounding), which can be fused into the kernel that produces the operand; here HPL runs a separate rounding kernel and the shim makes a copy, and both costs are included in the measured times.

Error model. For operands uniform on $[-1, 1)$ with both A and B zeroed, the relative error of $C = AB$ is

$$\|\hat{C} - C\|_F \,/\, \|C\|_F \approx \alpha\, 2^{-k}, \tag{1}$$

with $\alpha \approx 0.27$ for round-to-nearest, 0.38 for stochastic rounding and 0.70 for truncation ($k \geq 4$; truncation errs about 2.6 times more than round-to-nearest), independent of the inner dimension from 256 to 16384 (measured by splitting the error terms exactly in FP64). Truncation's error is almost entirely a systematic shrinkage: the least-squares slope of $\hat{C}$ against C departs from 1 by about $-0.6 \cdot 2^{-k}$ (-0.59 to -0.64 across k). This matters little for solvers that iterate to self-consistency or correct with a residual, but hurts processes that accumulate bias, which is why the rounding mode matters so much in Sections VI–VIII.

On real Llama2-70B layer-40 activations and weights, α = 0.22 (nearest) and 0.66–0.68 (truncation) for k = 2–4, within about 20% of uniform data. On four real FP64 GEMMs of Quantum ESPRESSO Si 1000, α is 0.19–0.32 (nearest) and 0.72–0.81 (truncation). On four real GEMMs of a gold surface, α is nearly constant across k within each GEMM, confirming the 2^{-k} law, but ranges from 0.07 to 9.6 (nearest) across GEMMs with the amount of cancellation: for nearly orthogonal overlaps such as $\langle\psi|\psi\rangle$ the result norm is far below the product of operand norms, amplifying the relative error. The form of the model holds broadly, but the prefactor must be measured per application; self-consistent iteration corrects these errors, which is why Quantum ESPRESSO tolerates 20–28 bits even with $\alpha \approx 10$.

Equivalent power saving. Under cap P, let the zeroed program reach performance $p'(P)$. Its equivalent power saving is

$$\Delta W(P) = P_{\text{native}}(p'(P)) - P, \tag{2}$$

where $P_{\text{native}}(x)$ is the cap at which the native program reaches performance x, linearly interpolated along the measured performance–cap curve. ΔW is the extra per-node power budget the unmodified program needs to match the zeroed one. If p' exceeds every native point we report a lower bound; if the GPU is not power-bound, the saving appears as lower measured power at the same speed. Because performance is concave in the cap, linear interpolation between widely spaced caps (e.g., 800 and 1000 W) slightly overestimates ΔW.

Tolerance-driven precision. Many solvers tighten their tolerance as they converge. In plane-wave DFT each Davidson diagonalization receives a threshold *ethr* that tightens from about 10^{-2} to about 10^{-13} during the self-consistent-field (SCF) iteration. We keep

$$k = \text{clamp}(\lceil -\log_2 ethr \rceil + m,\ k_{\min},\ 52) \tag{3}$$

fraction bits (margin m, floor $k_{\min} = 16$), so early iterations use fewer bits and the last ones use what the tolerance requires. This is an empirical rule, not a guarantee: *ethr* is an absolute eigenvalue tolerance (in Ry), not a relative mantissa error, and the margin absorbs the difference. It needs no application model, only a tolerance the application already computes.

V. EXPERIMENTAL SETUP

System. Fourteen nodes, each with one NVIDIA GH200 480GB superchip (72-core Neoverse-V2 Grace; Hopper GPU with 96 GB HBM3, maximum SM clock 1980 MHz, FP64 tensor-core peak 67 TFLOP/s [56]), connected by 200 Gb/s InfiniBand and managed by Slurm. The module power cap covers GPU, CPU and memory; the driver accepts caps from 200 to 1000 W (default 1000 W), while the datasheet lists 450–1000 W [56], so our caps below 450 W lie outside that range (measured power still follows them). The GPU alone has a separate 900 W limit. The default of 1000 W is also the highest cap, not an uncapped state: native HPL and FP64 GEMM still hit it much of the time. Software: NVHPC 26.5 (CUDA 13.2, cuBLAS 13.4, NCCL 2.29, HPC-X Open MPI 5), driver 615.71.

TABLE II
BIT COUNTS, ROUNDING AND SAMPLES PER EXPERIMENT.

Experiment	Bits k	Removal	Samples
GEMM, fill controls, crossover	fraction bits	truncation	GEMM: 3 nodes × 2 pairs; fills: 1 run; crossover: 2 rounds
HPL (8 nodes)	significand bits (36 = 35 fraction)	nearest, ties to even	2 node sets, 2 + 4 pairs per cap
HPL-MxP (8 nodes)	BF16 fraction bits	truncation	2 node sets, 2 + 4 pairs per cap
QE, PySCF, DF-MP2, Gaussian process, ExaGeoStat, MFEM, SW4, LAMMPS	fraction bits	nearest, ties away	QE: warm-up + 6 pairs; DF-MP2, GP: 6 rounds; others 2
LoRA	FP8 fraction bits	truncation (also nearest, stochastic)	2 seeds, 4 pairs
LLM inference	BF16/FP8 fraction bits	nearest, ties to even	6 models

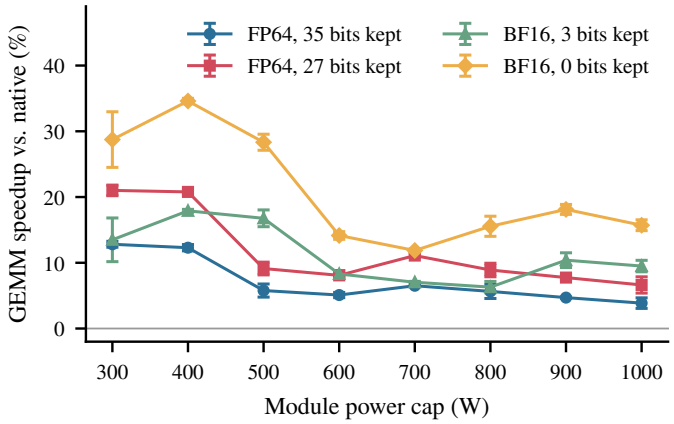


Fig. 1. GEMM speedup from mantissa zeroing under module power caps (FP64 32768×32768×2048, BF16 32768×32768×6144; 95% paired intervals, 3 nodes × ABBA).

Workloads. (1) cuBLAS GEMM microbenchmarks (FP64 32768×32768×2048; BF16 and FP8 E4M3 32768×32768×6144 with FP32 accumulation) on operands uniform on $[-1, 1)$. (2) HPL (a CUDA port of rocHPL, FP64, 8 nodes; tuned to $N = 286720$, NB 1024, a 2×4 grid, split fraction 0.05 and NCCL fixed at 8 channels with 2 MiB buffers) and HPL-MxP (BF16 LU, FP32 accumulation, FP64 iterative refinement, 8 nodes, $N = 491520$, on a diagonally dominant test matrix; Section VI-B). (3) Quantum ESPRESSO 7.6 GPU [57] (Si 1000-atom supercell at Γ, DGEMM; Au 112-atom surface with k points, ZGEMM), GPU4PySCF 1.8.1 [58], [59] (valinomycin B3LYP/def2-TZVP with density fitting; taxol RHF and DF-MP2), three non-GEMM FP64 codes (MFEM, SW4, LAMMPS), and a Gaussian-process likelihood (Section VII-E). (4) MLPerf Training Llama 2 70B LoRA with NVIDIA's v5.0 implementation (FP8 weights and GEMMs, activations offloaded to Grace memory), 8 nodes, global batch 8, and inference on six language models.

Implementation. Microbenchmarks and HPL-MxP zero operands in the kernel that produces them; HPL rounds both operands in a separate kernel before each FP64 DGEMM. For unmodified scientific software an `LD_PRELOAD` shim intercepts cuBLAS DGEMM/ZGEMM, GemmEx and strided-batched calls and CPU dgemm/zgemm, writes zeroed copies of A and B to scratch buffers (the caller's data are unchanged) and can record exponent histograms and per-row exponent spans of the operands. Quantum ESPRESSO is additionally modified to zero the Davidson vectors in place; DF-MP2 and the Gaussian-process code zero their operands in place; LoRA zeroes the frozen FP8 base weights once after loading. Table II lists what the bit count means, how bits are removed and how many samples each experiment has.

Measurement protocol. On every node nvidia-smi samples power, SM clock, temperature and throttle reasons every 250–500 ms. Every measurement starts from a GPU below 45 °C; within a cap the variants run in ABBA order (native, zeroed, zeroed, native) on the same nodes. Each sample records the module cap actually in effect, and windows whose cap differs from the intended one are discarded (another user's tool reset caps on the cluster at least twice; the affected measurements were rerun). Hardware power-brake events are read from cumulative throttle counters, enabled in both HPL sweeps and in a diagnostic HPL-MxP job, or from the sampled throttle reasons. The first native run of a job is often slower (warm-up), and since ABBA sequences usually start with native, the first pair can overstate the gain (Section VI-D), so the repeat runs start with one uncounted warm-up run. MLPerf comparisons always use the same seed, i.e., the same data order.

Power, energy and statistics. Energy integrates the driver's 1-s averaged module power over the measurement window, summed over nodes. Power in watts in tables and text is the mean of the instantaneous samples in a steady window; over 845 steady windows it agrees with the 1-s average within ±0.6%, but on bursty kernels (e.g., cuBLAS FP64 emulation) single instantaneous readings deviate by −37% to +41%, so energy does not use them. While the cap is hit, the 1-s average is often below the cap, so the limiter acts on a shorter, unknown time scale. Module power includes GPU, CPU and memory; under GEMM load the GPU accounts for 88–90% of it at 900–1000 W, falling to 77% at 400 W. Confidence intervals use paired differences: each "zeroed / native − 1" on the same node, cap and ABBA round is one sample, and we report 95% t intervals, which removes the 13–17% node-to-node clock spread. The GEMM microbenchmarks have 3 nodes × 2 pairs, Quantum ESPRESSO 6 pairs after a warm-up run, DF-MP2 and the Gaussian process 6 rounds from two jobs, and HPL and HPL-MxP 6 pairs per cap from two node sets (2 and 4). The intervals treat pairs from one job or node set as independent, which they are not fully, so they may be too narrow.

VI. RESULTS: GAINS UNDER POWER CAPS

A. GEMM

Zeroing speeds up GEMM at every cap, but the gain does not grow monotonically as the cap tightens (Fig. 1). FP64

TABLE III

HPL SCALED RESIDUALS ON 8 NODES AT 1000 W (THRESHOLD 16; REPEATED RUNS REPRODUCE EXACTLY; $N = 286720$ IS THE TUNED CONFIGURATION OF THE CAP SWEEP). BITS ARE SIGNIFICAND BITS INCLUDING THE HIDDEN BIT (36 BITS = 35 FRACTION BITS), ROUNDED TO NEAREST EVEN. (F): FAILS THE CHECK; BOLD: THE FAILURE THAT RULES OUT 28 BITS.

N	Native	36 bits	28 bits	24 bits	20 bits
131072	1.5×10^{-4}	0.16	13.97	456 (F)	9510 (F)
196608	1.3×10^{-4}	0.072	**18.09 (F)**	462 (F)	2850 (F)
262144	2.6×10^{-5}	0.042	14.70	91 (F)	599 (F)
286720	7.6×10^{-5}	0.038	15.40	—	—

keeping 27 bits is 9.1 ± 1.0, 8.1 ± 0.2, 11.1 ± 0.3 and 8.9 ± 1.0% faster at 500, 600, 700 and 800 W (95% intervals, 6 paired samples per point), an equivalent saving ΔW of 79–162 W; BF16 keeping 0 bits is 11.9–28.3% faster, ΔW from 121 W to above 200 W. The most energy-efficient cap also shifts: native FP64 peaks at 82 GFLOP/J at 500 W and FP64 keeping 27 bits at 90 GFLOP/J at 400 W; native BF16 peaks at 950 GFLOP/J at 600 W and BF16 keeping 0 bits at 1159 GFLOP/J at 500 W.

Against quantizing to FP8 (575 W, relative to native BF16), BF16 zeroed to 0 bits gives 1.15×, native FP8 E4M3 1.87× and FP8 zeroed to 0 bits 2.12×. On uniform data FP8 E4M3 errs by 3.6%, close to BF16 truncated to 4 bits (4.35%) or rounded to nearest at 3 bits (3.34%). On narrow-range data, including the real Llama 2 70B weights, FP8 wins in both speed and accuracy. On data whose rows span 2^{-24} to 2^{24}, however, FP8 flushes 63% of the values to zero (median elementwise error 100%) while the Frobenius error is only 3.9%; BF16 keeps every value's magnitude even at 0 bits.

B. HPL and HPL-MxP on 8 Nodes

The HPL residual check determines the usable bit count, and it must be checked at several sizes (Table III). From 36 to 28 bits the residual grows 86–410×, of the order of $2^8 = 256$, so 28 bits sits less than one bit from the threshold and is not an operating point, whereas 36 bits leaves a margin of about 100× (at least 98×, about seven bits). We first tuned native HPL at 1000 W. The configuration inherited from an H200 study ($N = 262144$, NB 1024, 2×4, default NCCL) gave 374.8–376.0 TFLOP/s; fixing NCCL at 8 channels with 2 MiB buffers (+4.5%) and raising N to 286720 (+2%) gave 390.2–398.8 TFLOP/s in three runs, and no neighboring setting of N, NB, split fraction or NCCL parameters beat this ≈2% run-to-run spread (24 settings, 26 runs; $f = 0.15$ and 4×2 grids were about 2 and 13% slower). We then swept the caps with this configuration on two node sets: set A (work4–11) with two ABBA pairs per cap, and set B (work3–8, 12 and 13), measured a day later after one uncounted warm-up run, with four pairs per cap. That day the second InfiniBand port of most nodes linked at only 2.5 Gb/s and NCCL and MPI routed part of the traffic over it (157 TFLOP/s on 8 nodes); pinned to the 200 Gb/s port, set B still reached only 365.5 TFLOP/s natively at 1000 W, 7% below set A, although its SM clock was higher (1961 versus 1936 MHz), so more of its time went to communication. Differences between the sets thus mix nodes and network state; every pair is matched within one job.

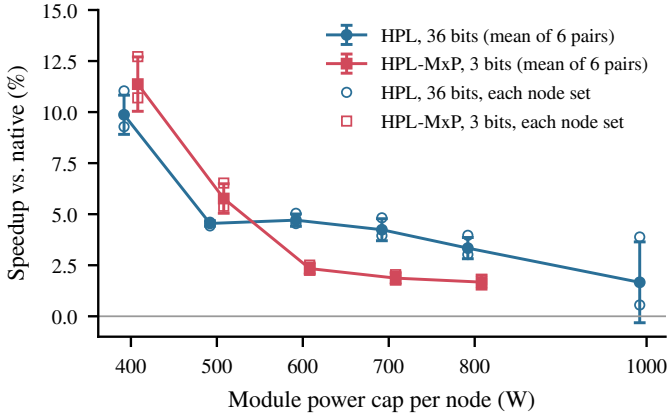


Fig. 2. 8-node HPL (36 significand bits, tuned, $N = 286720$) and HPL-MxP (3 kept bits, $N = 491520$, diagonally dominant test matrix) speedups: mean ± 95% t interval over 6 pairs from two node sets; open markers are each set's mean.

With 36 bits, HPL is 3.3–4.7% faster at 500–800 W over the 6 pairs (95% intervals of ±0.1–0.5 points; set A 4.0–5.0%, set B 3.0–4.6%; Table IV, Fig. 2), uses 4.1–4.6% less energy per solve, and is worth $\Delta W = 42$–74 W per node at 500–700 W (at 800 W the interpolation across the 800–1000 W gap overestimates). The speedups stay below the SM clock gains (4.5–7.0%), consistent with communication time that does not shrink with the clock, and set B, which communicates longer, gains less. Pairs within a job agree to within 0.4 (set B) and 0.6 points (set A), but the sets differ by up to 0.9 points at 600–800 W, so two pairs on one node set understate the uncertainty, and the pooled interval, which treats the 6 pairs as independent, may still be too narrow. At 400 W, below the specified range, the gain is 9.9 ± 1.0% (sets A/B: 11.0/9.3%) against only 6.3–6.6% more SM clock, for reasons we have not identified. At the default 1000 W cap both sets use less energy per solve (7.5/8.6%; pooled 8.3 ± 1.1%), but the speedup depends on how far the cap pulls the native clock down: native set A is capped in 75% of the samples at 1936 MHz and also thermally throttled in 15% of them (36 bits: 5%; no other cap or set throttles), and the 36-bit run is 3.9% faster, partly a temperature effect, whereas native set B averages 1961 MHz, 1% below the maximum, and gains only 0.6%. Tuned native HPL reaches 393.9 TFLOP/s at 1000 W on set A (mean of two runs; 398.8 TFLOP/s at best during tuning), about 73% of the 8 GPUs' FP64 tensor-core peak, and the 36-bit run 409.2 TFLOP/s (research results, not official scores). The untuned configuration gave 3.4–8.3% at 400–800 W and 0.7/7.7% at 1000 W. (On set A, 28 bits is 5.7–16.3% faster at 400–800 W, but its residual reaches 15.4 at $N = 286720$; we list it only as a boundary reference.)

HPL-MxP (BF16 LU with FP64 refinement) zeroes the BF16 operands whenever they are written back. Our runs use a test matrix whose diagonal entries equal the sum of the magnitudes of the off-diagonal entries in their row

TABLE IV

DELIVERED PERFORMANCE ON 8 NODES IN TFLOP/S, NODE SET A / B (MEANS PER SET). HPL: TUNED, $N = 286720$; HPL-MxP: $N = 491520$, REPORTABLE RATE INCLUDING REFINEMENT. THE 8 GPUS' FP64 TENSOR-CORE PEAK IS ABOUT 536 TFLOP/S (8 × 67 TFLOP/S [56]).

	HPL (FP64)		HPL-MxP (BF16 LU)	
Cap	Native	36 bits	Native	3 bits
400 W	225.0/228.0	249.9/249.2	2076/1922	2340/2128
500 W	288.0/278.2	300.7/291.0	2823/2555	3007/2692
600 W	318.0/304.5	334.0/318.3	3229/2858	3310/2922
700 W	346.2/328.3	362.9/341.3	3449/3039	3519/3094
800 W	370.8/345.8	385.6/356.3	3655/3178	3720/3229
1000 W	393.9/365.5	409.2/367.5	—*	—

Set A: work4–11 (HPL), work4–7 and 10–13 (HPL-MxP), 2 pairs per cap; set B: work3–8, 12, 13, 4 pairs. *Native HPL-MxP trips the chassis power brake at 1000 W on set A; set B was not run there.

TABLE V

QUANTUM ESPRESSO WITH IN-PROGRAM TRUNCATION (ROUND TO NEAREST) OF DAVIDSON BLOCKS: ONE UNCOUNTED WARM-UP RUN, THEN MEAN ± 95% t INTERVAL OVER 6 INTERLEAVED PAIRS; NATIVE ROWS GIVE THE MEAN OF THE 6 NATIVE RUNS. ENERGY INTEGRATES 1-S AVERAGED MODULE POWER.

System	Setting	Time	Energy	ΔE (Ry)	SCF
Au 112 (450 W)	Native	91.4 s	35.2 kJ	—	28
	Adapt. m=0	$-3.6 \pm 0.2\%$	$-3.4 \pm 0.1\%$	$< 10^{-8}$	28
	Fixed 24	$-2.0 \pm 0.2\%$	$-2.1 \pm 0.1\%$	3×10^{-8}	28
Si 1000 (550 W)	Native	205.7 s	108.4 kJ	—	24
	Adapt. m=0	$-2.1 \pm 0.1\%$	$-2.2 \pm 0.1\%$	$< 10^{-8}$	24
	Fixed 28	$-2.0 \pm 0.1\%$	$-2.1 \pm 0.2\%$	$< 10^{-8}$	24

(diagonal dominance), factor the diagonal tiles approximately (`--approx-diag`), and refine with Richardson iteration and an FP32 preconditioner. The HPL-MxP rules allow such diagonally dominant matrices "for testing purposes only" [60], and on such a matrix even coarse L and U factors precondition well, so the stability reported here may be a property of this matrix and cannot be assumed for the generators allowed in official runs. Like HPL, HPL-MxP has a failure boundary, and we use 3 kept bits, stable at all four sizes $N = 245760$–491520 (per-run residuals 1.3–3.8, always 3 refinement iterations). We measured it on two node sets as well (Table IV): set A with two pairs per cap and set B, the second HPL set with the IB ports pinned, with four pairs after a warm-up run. Its BF16 LU is fast, so communication weighs more than in HPL, and set B's native rate is 7–13% below set A's. At $N = 491520$, over the 6 pairs, 3 bits is $5.8 \pm 0.7\%$ faster at 500 W (sets A/B: 6.5/5.4%) and 1.7–2.3% faster at 600–800 W (intervals of ± 0.3 points; the sets differ by only 0.2–0.3 points there), using 3.6–5.9% less energy per solve; at 400 W it is $11.4 \pm 1.3\%$ faster. ΔW is 34–45 W at 500–700 W on both sets (at 800 W only set A has a native point above the cap, from another job; we do not report it). Refinement grows from 1 iteration (native) to 3 at every cap, while the speedup of the LU itself shrinks from 13.5–16.0% at 400 W to 5.2–6.0% at 800 W, so above 600 W only about 2% remains after the extra refinement. Keeping 0 bits (set A) gives 7.1–26.0% (ΔW 72–146 W) at $N = 491520$ with per-run residuals of 10.0–11.1 (a margin of only about 1.5×); at $N = 393216$ its residual reaches 11.7–13.5, and at $N = 245760$ refinement jumps from 1 to 52 iterations and performance falls by 70%: passing the residual check does not imply being faster. With fans at full speed, native HPL-MxP at 1000 W trips the chassis hardware power brake (4–10 s each time; in a diagnostic job the brake flag appears in every native run and in no 0-bit run), so we exclude 1000 W from speedups.

C. Plane-Wave DFT and Quantum Chemistry

Plane-wave DFT is tolerant. With 28 kept bits, the total energies of QE Si 1000 and Au 112 move by 1.0×10^{-5} and 2.3×10^{-5} Ry under truncation and by 3.5×10^{-7} and 2.8×10^{-7} Ry under round-to-nearest. The gold surface still converges at 20 bits with nearest rounding (5.5×10^{-5} Ry, both runs); for Si 1000, 20 bits is the failure edge (one run converges with 1.1×10^{-4} Ry, the other aborts at iteration 24), and truncation diverges or slows at 20 bits. Gaussian-basis quantum chemistry needs more bits. GPU4PySCF valinomycin DFT errs by 1.1×10^{-8} E_h at 44 bits; at 36 bits the energy oscillates near 10^{-8} E_h and cannot meet a 10^{-10} convergence threshold, but with the chemically sufficient 10^{-7} threshold 40, 36 and 32 bits converge in the same 12 iterations as native, with a 32-bit nearest error of 1.2×10^{-5} E_h (≈0.007 kcal/mol). At 36–40 bits the error is then set mainly by the threshold (32 bits errs an order of magnitude more), consistent with the ≈37 significant bits found sufficient by Dawson et al. [51].

Under caps, with the shim truncating every GEMM and compared against a baseline that takes the same copy path without truncating, Si 1000 at 550 W with 28-bit nearest rounding is 6.2% faster and uses 6.4% less energy (SM clock 1348→1446 MHz), and Au 112 at 450 W is 4.9% faster with 5.1% less energy (one truncated run each, between baseline runs before and after). These are upper bounds for a library that truncated internally; against the unmodified program the shim is 0.4% slower (Au 112, 450 W) because of its ≈5% copy cost. The in-program version of Section VI-D truncates only Davidson blocks without copies and is the deployable number.

For GPU4PySCF, nsys shows FP64 GEMM kernels at 46% of GPU kernel time and 32% of total runtime for valinomycin DFT (31% and 27% of kernel time for taxol and tamoxifen); the rest is dominated by copies of computed intermediates (36–50%) and integrals. With 32–36 bits needed and FP64 GEMM keeping 35 bits 5.1–6.5% faster at 500–800 W, truncation alone is bounded at about 32% × 5–6.5% ≈ 1.6–2.1% end to end (3.9% at 400 W). This estimate assumes the default grid and GEMM time proportional to flops at 1000 W speed; on a finer grid (level 6), valinomycin measured at 450–550 W is 3.3–3.5% faster (Section VII-D). Neither number implies that truncation beats INT8 emulation here: the two dominant GEMM shapes favor different methods (Section VII-B), and choosing by shape bounds the gain at about 3.5% with cuBLAS emulation for the long-k shape, or 6.6% with the less accurate GEMMul8.

D. Tolerance-Driven Bit Counts in Quantum ESPRESSO

The adaptive rule truncates the wavefunction blocks before and after applying the Hamiltonian in the Davidson diagonalization. It reproduces the native total energy in every printed digit (10^{-8} Ry) on both systems with the same number of SCF iterations, while cutting time by 2.1–3.6% and module energy by 2.2–3.4% (Table V); k rises from 16 to 39 on the gold surface and to 42 on Si 1000 as the SCF converges. Each job starts with one uncounted warm-up run: in two earlier jobs on other nodes without it, the first run (a native one) was slower, inflating the gain of the pair containing it. Those jobs gave 4.0% (gold) and 2.4% (Si) after dropping each job's first pair; the 0.3–0.4-point difference from the new runs exceeds either interval and reflects node-to-node variation, so we report the warm-up runs. A margin of $m = 2$, run once in the earlier jobs, gave −3.7 and −1.6%. The energies stay unchanged for three reasons. SCF is a self-correcting fixed-point iteration that already solves early iterations to a loose *ethr*, and the rule keeps the truncation error roughly within what the current Davidson step already tolerates, so later iterations correct it. In the last iterations $ethr \approx 10^{-12}$–10^{-13} and $k \approx 39$–42, and the total energy is second order in the wavefunction error. And 10^{-8} Ry on a total of about 10^{4} Ry is a relative 10^{-12}. Fixed 24 bits is coarser than the threshold late in the SCF and leaves 3×10^{-8} Ry.

On the gold surface the adaptive rule beats fixed 24 bits in all 6 pairs by 1.2–1.8 points with no energy error, but not only through power. It makes Davidson do about 7–9% less work (311 versus 336 calls of the Hamiltonian application), and in one run whose cap had been externally reset to 1000 W, where the code is nearly unconstrained (native 1957–1960 MHz, both truncated variants about 1973 MHz), it was still about 2% faster than native and fixed 24 bits (73.5–73.7 versus 74.8–75.1 s), close to its 1.6-point lead over fixed 24 bits at 450 W. About half of the gold-surface gain thus comes from fewer Davidson iterations, for reasons we have not identified. On Si 1000 the work is unchanged (206 versus 205 calls), so its 2.1% can be read as a power effect, and the rule ties with fixed 28 bits (0.05 ± 0.11 points). Its main value is that no safe bit count must be known in advance: one margin leaves both systems unchanged and matches or beats the best fixed count, which can only be found after the fact. Unlike Woo et al. and Huang et al., the arithmetic stays on native FP64 units with one-bit granularity, the signal is the Davidson threshold, and the gain comes mainly from clock headroom under a cap (plus, on the gold surface, fewer iterations) rather than a higher-throughput format.

E. A Power-Hungry Application: DF-MP2

The codes above do not fill a 1000 W cap on average (Quantum ESPRESSO: about 795–828 W for Si 1000 and 535–600 W for the gold surface). The density-fitted MP2 module of GPU4PySCF [58] draws more: during MP2 the GPU draws about 800 W and the module 869–876 W, although at 1000 W only 24–53% of the samples hit the cap, and matrix multiplication takes 78% of GPU kernel time, nearly all of it the contraction $g_{ab} = B_i B_j^{\mathsf{T}}$ over slices of the three-index tensor B (FP64 DGEMM whose inner dimension is the auxiliary basis). We zero B in place once after it is built, so every later contraction benefits without copies, and compare with GPU4PySCF's built-in FP32 path and cuBLAS emulation. One process runs DF-RHF once and then the variants interleaved (taxol def2-TZVP, 113 atoms, 2228 basis functions; six rounds per cap from two jobs on two nodes; the second ran an uncounted warm-up at each cap first).

TABLE VI

DF-MP2 (taxol def2-TZVP): speed relative to native (mean ± 95% t interval over 6 rounds), and the error $|\Delta E|$ of the MP2 correlation energy in E_h (truncated variants round to nearest). At 1000 W the truncated variants run at about native speed and draw 61–75 W (32 bits) and 106–127 W (20 bits) less, using 8.4% and 14.3% less energy.

Cap	Native	32 bits	20 bits	FP32	Emu. 32
1000 W	39.5 s	$+0.5 \pm 0.2$%	$+0.8 \pm 0.1$%	+9.8%[a]	−35%
700 W	43.9 s	$+4.9 \pm 0.4$%	$+7.4 \pm 0.4$%	+17.9%	−28%
500 W	51.5 s	$+4.7 \pm 0.2$%	$+7.7 \pm 0.5$%	+14.6%	−18%
$\lvert\Delta E\rvert$	—	1.0–1.6×10^{-11}	1.2×10^{-8}	2.8–5.7×10^{-8}	2.2×10^{-8}

[a] Mean of six rounds; one was only 2.6% faster, the others 10.9–11.5%.

TABLE VII

FP64 kernels without tensor cores: speedup at 400–700 W with 36 and 20 kept bits, and module power saved at 1000 W with 20 bits.

Kernel	36 bits	20 bits	Saved
FMA loop	+2.7–4.8%	+5.6–9.0%	138 W
N-body, inputs only	+1.3–2.0%	+2.6–3.9%	59 W
N-body, also in loop	—	−7.7 to −8.7%	60 W
Memory copy (450 W)	+7.7%	+14.0%	70 W
CG, matrix values	≤+1.7%		7 W

Keeping 32 bits changes the MP2 correlation energy by about 10^{-11} E_h and 20 bits by 1.2×10^{-8} E_h (Table VI); under caps the gains are on par with HPL. The built-in FP32 path is faster still (one of its six rounds at 1000 W was much slower), and its error of a few 10^{-8} E_h is within chemical accuracy: the tensor's exponent span is narrow (27 binades, i.e., powers of two, overall and a median of 18 within a row), so FP32 fits (Section VII-D). cuBLAS emulation errs by 2.2×10^{-8} E_h but is 18–35% slower than native at this shape ($m = n = 2002$, $k = 5477$).

F. FP64 Beyond GEMM

Ozaki-type emulation applies only to dense matrix multiplication, so we also measured FP64 work that bypasses the tensor cores (Table VII). The benefit depends on how many operations separate the truncation point from the multiplier and on what share of the data traffic is truncated. In the FMA loop the inputs feed the multiplier directly; in N-body the position differences regrow full mantissas, and masking the intermediates inside the loop costs more issue slots than it saves; a copy speeds up only when its bandwidth is power-capped; and in CG the vectors dominate the traffic and the indices cannot be truncated. CG keeps the same 490 iterations, with a true residual of 4×10^{-9} at 36 bits.

Third-party codes. We zeroed, once at setup, the coefficient arrays that are stored and multiplied in every step in three GPU FP64 codes: MFEM's partial-assembly quadrature data (about 43% of CG memory traffic), SW4's material parameters (about 9%) and LAMMPS EAM tables (cache resident). Without zeroing all three are bitwise identical to the originals or within rerun noise. MFEM (38.27 M unknowns, $p = 3$) is 0.9/2.0% faster with 36/20 bits (nearest) at 450 W and 0.7/1.4% at 550 W, unchanged at 1000 W; the solution changes by at most 2.3×10^{-11}. SW4 (123 M grid points) is 0.6/1.3% faster at 350 W and 0.3/0.5% at 450 W, with receiver waveforms changing by at most $3.4\times10^{-11}/3.4\times10^{-6}$ (doubling the grid changes them by 4–10%); LAMMPS EAM (16.38 M atoms) moves 0.0–0.4%, within noise. The gain order, MFEM ≥ SW4 > LAMMPS, follows the coefficients' share of memory traffic: kernel-level gains of 4.8–14.0% at 450–500 W are diluted by the rest of the full-mantissa computation. Zeroing fits best when inputs flow directly and in volume into multipliers or data paths.

VII. When Format Changes Win and When They Fail

A. Range versus Precision

From the exponent histograms recorded by the shim (central 99.9%), operands span 12–14 binades in HPL, about 13 in Llama 2 70B weights, 71–93 in Quantum ESPRESSO, 55–64 in GPU4PySCF taxol and 371–690 in valinomycin (down to 2^{-691}, beyond even FP32's range). Every scientific code fills the FP64 mantissa, so how many bits are needed must be found by zeroing experiments, or estimated from the bitwise information content of the data [34]. Placing range against needed bits, plane-wave DFT (a range too wide for FP16 or FP8, modest precision) is where mantissa zeroing has no smaller-format alternative, Gaussian-basis quantum chemistry needs both range and precision, and LLMs and HPL have narrow ranges and can move to smaller formats.

B. Crossover with INT8 Emulation of FP64

cuBLAS 13 includes an Ozaki-type FP64 fixed-point emulation with a settable number of mantissa bits. On the same node and cap we compare truncated native FP64 with emulation at similar error (32768×32768×2048, 400–1000 W, ABBA). Emulation works in 8-bit slices, $\lceil (b+1)/8 \rceil$ of them for b bits: 24 and 28 bits both use four and give identical results (2.3×10^{-9}), and 32 and 36 bits both use five. On this square-ish shape, three slices (5×10^{-7}) beat truncation to 20 bits by 51–73%; four slices (2×10^{-9}) beat 28 bits by 6% at 1000 W to 23% at 400 W; at 10^{-11} truncation (about 36 bits) is 11–28% faster than five slices; and at 10^{-13} and below truncation is more than 40% faster. The crossover is thus near 28–32 needed bits, and on this shape lower caps favor emulation (the caps were measured on different nodes, but the single-node data of Table IX show the same trend). These comparisons truncate toward zero; the applications round to nearest, which zeroes the same bits for one extra addition and errs about 2.6 times less (about 1.4 bits), so at equal accuracy rounding needs about one bit fewer and the crossover moves slightly toward truncation; we did not repeat the sweep with rounding. GEMMul8 is version 3.5.1 (September 29, 2026; the later 3.5.2 only fixes HIP compilation).

TABLE VIII
The two GEMM shapes that each carry 35% of valinomycin DFT flops: tall-skinny 757248×300×2958 and long-k 2958×2958×76800 (TFLOP/s; relative error in parentheses; GEMMul8 errs by 1.0×10^{-10} and 8.2×10^{-10} on the two shapes, 10–80 times more than truncation to 36 bits).

	Tall-skinny		Long k	
Method	1000 W	575 W	1000 W	575 W
FP64 native	51.6	36.8	50.6	36.2
Truncation, 36 bits (1.0×10^{-11})	54.0	38.9	53.7	38.0
cuBLAS emu., 32 bits (9.7×10^{-12})	19.5	16.6	65.4	46.0
cuBLAS emu., 28 bits (2.3×10^{-9})	24.2	20.9	89.9	65.6
GEMMul8, 10 moduli	17.9	16.7	102	75.6

The crossover depends on shape. Intercepting all 45,574 GEMMs of the valinomycin DFT run reveals two shapes that each carry 35% of the flops (Table VIII). With $n = 300$, emulation cannot amortize its splitting and scaling: even at 28 bits it runs at 0.47–0.57× native speed, and truncation wins at every accuracy. With $k = 76800$ the fixed costs amortize: emulation is still 21–22% faster than truncation at the 10^{-11} level, and GEMMul8 (errors 1.0×10^{-10} and 8.2×10^{-10} on the two shapes) is nearly twice as fast at 10^{-9}. Within one application the best choice is per call: truncation for tall-skinny shapes, emulation for long k.

A shape sweep and a predictive model. We fixed $m = 8192$ and swept $n \in \{256, 1024, 4096, 8192\}$ and $k \in \{256, 1024, 4096, 16384, 81920\}$ at 1000, 575 and 450 W, two rounds each (480 records; Fig. 3). Truncation to 36 bits beat native in all 60 shape–cap combinations (3.8–11.9% at 450 W, 3.6–5.9% at 575 W, 0.5–4.8% at 1000 W). cuBLAS emulation with 32 bits wins roughly when $n \geq 4096$ and $k \geq 16384$ (and at $n = 1024$, $k = 81920$ at 1000 and 575 W) (at best 0.74× truncation time at 1000 W), and its winning region shrinks at lower caps: 5 of 20 shapes at 1000 and 575 W, 2 at 450 W. Where the emulation itself is not power-bound, however, lower caps favor it: at $n = 8192$, $k = 4096$ its time relative to truncation falls from 1.17 to 1.08 to 1.05 as the cap goes from 1000 to 575 to 450 W. The direction thus depends on whether the emulation itself is power-limited. The four values of n were measured on different nodes: each ratio is paired on one node, but shape effects and node differences cannot be separated, a flaw of the design rather than a mere caveat. GEMMul8 with 10 moduli needs only 0.47–0.80× the truncation time when $n, k \geq 4096$ (and is slightly faster at $n = 1024$ with $k \geq 16384$), at an order of magnitude larger error ($\approx 10^{-10}$ versus 10^{-11}), and is up to 4.5× slower when n or k is small. Emulation's extra work is input splitting ($\propto mk + nk$) and output accumulation ($\propto mn$); dividing by the mnk multiply gives

$$T_{\text{emu}}/T_{\text{trunc}} = A + B\,(1/m + 1/n) + C/k, \tag{4}$$

where A is the ratio of per-flop speeds. Least-squares fits per cap (about 20 points for three parameters, so R^2 is only

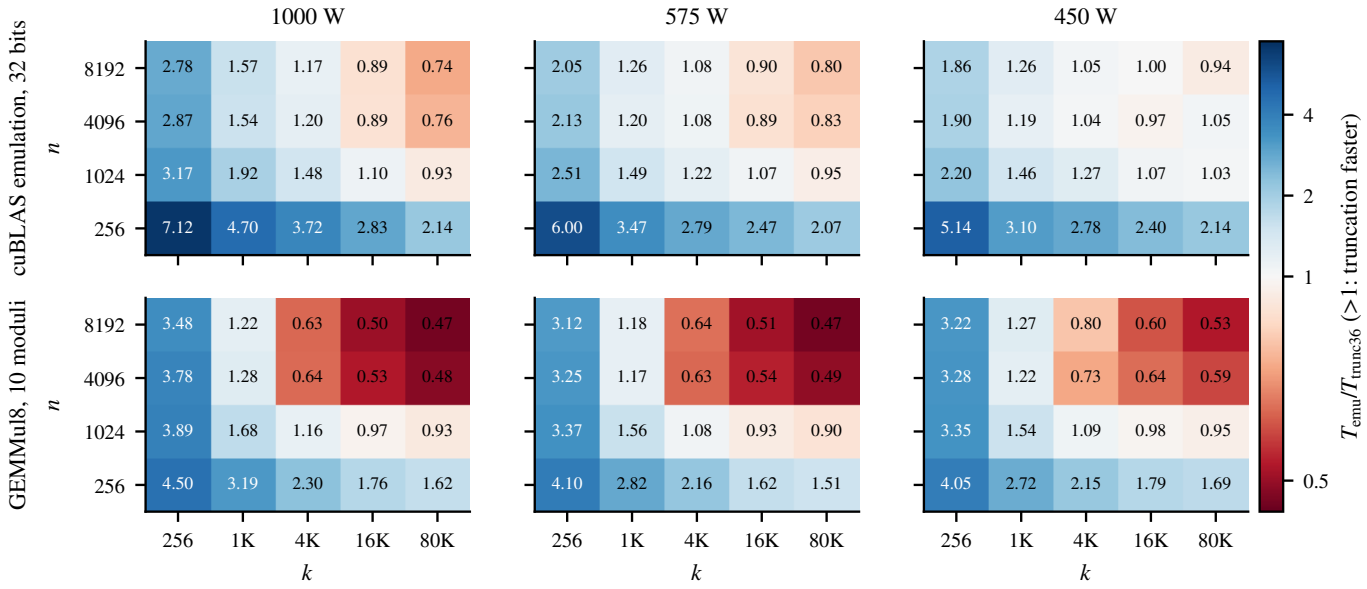


Fig. 3. Time of INT8 emulation relative to truncation to 36 bits, $T_{emu}/T_{trunc36}$ (m = 8192; blue: truncation faster). Top: cuBLAS emulation with 32 bits (error comparable to truncation, $\approx 10^{-11}$); bottom: GEMMul8 with 10 moduli (about ten times larger error, $\approx 10^{-10}$); columns: module caps. Means of two rounds.

TABLE IX
SQUARE GEMM (32768×32768×2048) IN TFLOP/S BY CAP ON ONE NODE, NATIVE FP64 FIRST AND THE OTHER METHODS ORDERED BY RELATIVE ERROR.

Method (error)	1000 W	575 W	450 W
FP64 native	59.2	43.2	35.8
Trunc., 36 bits (1.0×10^{-11})	61.8	45.5	38.6
GEMMul8, 10 moduli (1.0×10^{-10})	70.8	51.4	34.7
GEMMul8, 9 moduli (1.6×10^{-9})	78.2	56.6	38.2
cuBLAS emu., 28 bits (2.3×10^{-9})	67.8	55.2	47.5
Trunc., 28 bits (2.6×10^{-9})	63.2	46.6	39.6
FP32 SGEMM (8.1×10^{-7})	51.2	40.4	34.6
3xTF32 (4.8×10^{-6})	96.6	72.0	56.4

descriptive) reach R^2 = 0.98–0.99 for GEMMul8, versus 0.86–0.89 for a single effective size $1/(1/m + 1/n + 1/k)$, so for GEMMul8 input and output costs must be separated; cuBLAS emulation reaches only 0.88–0.89, no better than the single-size model, the residual likely reflecting different internal kernels per shape. On the two held-out valinomycin shapes (two caps × two methods), the model picks the winner in 8 of 8 cases, with ratio errors of 6–15% for cuBLAS and 24–37% for GEMMul8 (m = 757248 lies far outside the fitted range). All eight cases lie far from the crossover (emulation takes 0.5–0.8× or more than twice the truncation time), so this is a loose test, and with m fixed at 8192 in the fit the dependence on m is untested. The model is therefore an empirical description of these measurements; using it to choose a method per call needs tests near the crossover, at other m and with all shapes on one node. At the GEMM level, with truncation in the kernel that produces the operands, truncation was never slower than native in these 60 shape–cap combinations.

The power of the emulation itself changes the winner. GEMMul8 draws about 970 W at a 1000 W cap and at 450 W its clock drops to 650–720 MHz, where truncation becomes faster; cuBLAS fixed-point emulation with 28 bits draws only about 706 W at 1000 W and gains relative ground at lower caps on the square shape (Table IX). Which way a tighter cap tips the balance thus depends on how much power the emulation itself draws. Truncation is faster than native at every cap, though its gain does not grow monotonically as the cap tightens (Section VI-A).

C. FP32 and 3xTF32: Accumulation, Not Range

3xTF32 (our own implementation, in the shim and in a benchmark script, not the CUTLASS one) is the fastest method in Table IX but has only FP32-level accuracy, and its error grows with the inner dimension: on 4096×4096 it is 6.4×10^{-7}, 4.8×10^{-6} and 3.8×10^{-5} at k = 256, 2048 and 16384, whereas FP32 SGEMM grows as $\sqrt{k}$ (2.9×10^{-7}, 8.1×10^{-7}, 2.3×10^{-6}) and FP64 truncation is independent of k. The measured log slope of about one (7.5–7.9× per 8× in k) points to a systematic error in TF32 tensor-core accumulation rather than random rounding, consistent with the round-toward-zero accumulation Fasi et al. found on V100, T4 and A100 [61] (bit-accurate Hopper models: [62]). At k = 2048 its error is 18 times that of FP64 rounded to nearest at 20 bits (2.6×10^{-7}) and 7 times that of FP64 truncated to 20 bits (6.7×10^{-7}).

Quantum ESPRESSO's operands fit in FP32's range, so we replaced every FP64 and complex FP64 GEMM with single precision through the shim (inputs to FP32; 3xTF32 additionally splits into big and small parts multiplied three times on TF32 tensor cores; C combined in FP64). On the gold surface at 450 W (2 interleaved runs each), FP32 aborts at SCF iteration 26 and 3xTF32 and TF32 at iteration 2 when the Davidson bands fail to converge, and the integrated

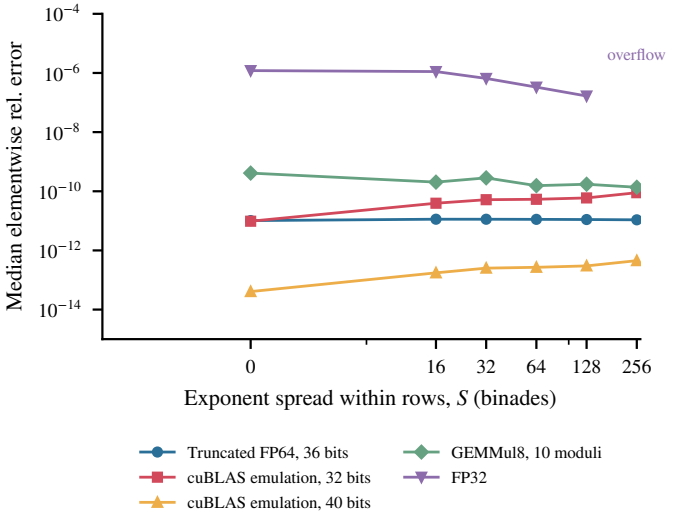


Fig. 4. Synthetic exponent spread: elements of 8192×8192×16384 operands have exponents uniform over S binades within each row (1000 W).

charge drifts from 1232 electrons (FP32 +0.012, 3xTF32 +0.074); FP64 rounded to 24 and 20 bits differs from native by 8.1×10^{-6} and 5.5×10^{-5} Ry and converges in 28 iterations. Si 1000 at 550 W behaves the same: FP32 aborts at iteration 14 and 3xTF32 at 9, with 4000.18 and 4001.52 electrons instead of 4000. The difference is accumulation: truncated FP64 still accumulates in FP64, while Quantum ESPRESSO's inner dimension is the number of plane waves (95386 on the gold surface plus a real GEMM with $k = 4.3\times10^{6}$; 375286 for Si 1000), where 3xTF32 already errs by 5.6×10^{-5} to 2.2×10^{-4}, the same order as plain TF32. FP32 GEMM on Hopper is also slower than FP64 tensor cores (on the gold surface about 1.5× per SCF iteration, estimated). Replacing every GEMM with single precision thus fails for this long-inner-product solver even though the range fits; mixed precision that uses FP32 only in selected steps, as Woo et al. do [49], is a different matter. FP64 truncation is the reduced-precision choice here that leaves the algorithm unchanged.

D. Exponent Span Limits Format Changes

Truncation keeps every element's 11-bit exponent and changes only its mantissa; format changes must handle the exponent. Per-tensor scaled formats (FP32, FP8) need the whole tensor inside the format's range, and Ozaki-type INT8 emulation slices each row (column) relative to its maximum, so with k kept bits elements more than 2^{-k} below the row maximum are lost. The exponent span of the data therefore limits format changes, whereas truncation is unaffected.

Synthetic data. On 8192×8192×16384, a shape where emulation is faster at narrow spans, we draw element exponents uniformly over S binades (1000/575 W, two rounds; Fig. 4). Truncation to 36 bits keeps a median elementwise error of $1.0\text{–}1.1\times10^{-11}$ and a speedup of 4.4–6.3%. cuBLAS emulation with a fixed 32 bits degrades by 4–9× in median and 5–19× at p99 (S = 16–256), the equivalent of 2–3 fewer bits,

TABLE X

Real data (1000 W): flop-weighted fraction of nonzeros more than 2^{-32} below their row maximum, and the total-energy error of each method. The "Nearest" rows keep 36 or 32 bits with round-to-nearest; cuBLAS emulation with 36 and 32 bits uses five slices, with 40 bits six; auto: its automatic mode (at most 79 bits). The Au 112 fraction comes from a two-iteration benchmark input, its errors from the fully converged input.

	Valinomycin	QE Si 1000	QE Au 112
Below 2^{-32}	24–44%	0.24–0.54%	0.008–0.02%
Nearest, 36 bits	9.6×10^{-7} E_h	0 Ry	0 Ry
Nearest, 32 bits	1.2×10^{-5} E_h	0 Ry	4×10^{-8} Ry
cuBLAS emu., 36/32	$\mathbf{5.25\times10^{-2}}$ $\mathbf{E_h}$	6×10^{-8} Ry	5×10^{-8} Ry
cuBLAS emu., 40	8.4×10^{-8} E_h	—	—
cuBLAS emu., auto	4×10^{-9} E_h	—	—
GEMMul8, 10 moduli	diverges	out of memory	3.7×10^{-7} Ry

which costs a whole extra 8-bit slice to recover. Its dynamic mode, which targets FP64-equivalent accuracy, raises the bit count from 58 to 301 and falls back to native FP64 beyond its maximum, but with the extra work of determining the span it takes 1.6–2.6× the truncation time (2.6× at S = 16); FP32 overflows at S = 256. GEMMul8 is barely affected on uniformly spread data. If the spread is only between rows (one scale per row), emulation is unaffected while FP32 still overflows.

Real data. The shim samples 256 rows of op(A) and 256 columns of op(B) per GEMM and records the fraction of nonzeros more than 2^{-32} below their row maximum, weighted by flops (validated against theory on synthetic data). In Gaussian-basis DFT the atomic-orbital values on the grid decay exponentially with distance, and in the worst shape 87% of elements are more than 2^{-32} below their row maximum. There, fixed 36-bit cuBLAS emulation (five slices, like 32 bits) lets the SCF converge normally in 12 iterations to a total energy that is off by 52.5 mE_h (≈33 kcal/mol), and GEMMul8 diverges; rounding to the same 36 bits errs by 9.6×10^{-7} E_h (Table X). What fails is the user-fixed bit count. With 40 bits (six slices) the emulation is correct (8×10^{-8} E_h) but takes about 1.5× the time of truncation to 36 bits (see below), and cuBLAS's automatic mode, capped at 79 bits, is correct too (4×10^{-9} E_h): 89% of the flops would need 80 or more bits and fall back to native FP64, the rest is emulated with 72–79 bits, and the run takes 2.1× the native time. On wide-span data emulation is therefore not impossible, but not worthwhile. HPL sits at the other extreme: only 2.7×10^{-6} of its flop-weighted elements are more than 2^{-32} below their row maximum, and fixed 36-bit emulation passes the residual check (0.197, against 0.057 for HPL's own rounding to 36 bits) but runs at 0.65× native speed on one GPU (N = 81920; the automatic mode keeps the native residual level, 1.9×10^{-6} against 1.7×10^{-6}, at 0.35×). On Quantum ESPRESSO, with narrow per-row spans, fixed 36-bit emulation errs by only 5–6$\times10^{-8}$ Ry. Uniform synthetic spreads understate the problem, since in real data many elements of a row lie far below its maximum. A wide per-row span, which the shim measures cheaply beforehand (Fig. 5), is thus a risk indicator: a necessary but not sufficient

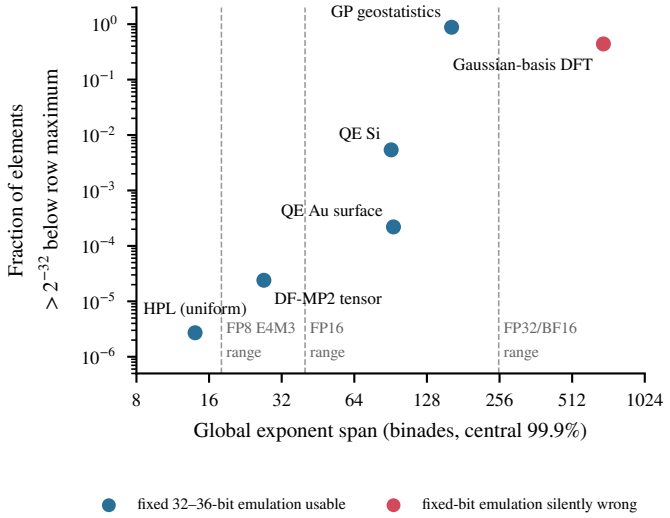


Fig. 5. Global exponent span (central 99.9%) against the flop-weighted fraction of elements more than 2^{-32} below their row maximum, with the measured outcome of fixed 32–36-bit cuBLAS emulation (e.g., HPL: residual 0.197, passes; Gaussian-basis DFT: 52.5 mE_h error; Gaussian process: global span 162, estimate unchanged). All points are measured; coordinates and their sources are listed in `gh200/results/range_map_points.csv`.

condition for failure, and tile-level mixed precision can also handle such data (Section VII-E). DF-MP2 (Section VI-E) sits at the narrow end, where FP32 works.

Wide range, partly power-bound DFT. These DFT runs draw only 650–690 W at 1000 W, but are power-bound below that. For valinomycin (grid level 6) at 550 and 450 W, rounding to 36 bits through the shim is 3.3–3.5% faster than the copy baseline with 2–3% less energy and an error of 9.5×10^{-7} E_h; emulation must keep 40 bits to stay correct, which is then about ten times more accurate (8.4×10^{-8} E_h) but takes 1.48–1.53× the baseline time (two rounds each).

E. Wide Range and Power-Bound: Gaussian-Process Geostatistics

Maximum-likelihood estimation of a Gaussian process in geostatistics factors an $n\times n$ Matérn covariance matrix by Cholesky at every likelihood evaluation. As in HPL, almost all work is FP64 matrix multiplication, but the covariance decays exponentially with distance. We implemented an ExaGeoStat-like kernel: $n = 80000$ random 2-D locations, Matérn-3/2 covariance (nugget 10^{-4}), blocked Cholesky ($n_b = 1024$, trailing updates by cuBLAS DGEMM), and the log-likelihood at six range parameters (0.02–0.08). Each panel L_{21} keeps k bits in place before the trailing update, in one fused pass that walks the panel in memory order (an earlier version with two separate, uncoalesced passes left 28 bits at 5.2–5.7% under caps, so the reduction itself must be cheap). We compare with cuBLAS emulation and FP32 (1000/700/500 W, six interleaved rounds from two jobs on the same node; the second ran an uncounted warm-up at each cap first). Native draws 953–970 W of module power at 1000 W and is capped 91–94% of the time; the sampled panel rows span 162 binades (central 99.9%), and 88% of the panels' nonzeros are more than 2^{-32} below their row maximum.

TABLE XI
GAUSSIAN-PROCESS LIKELIHOOD ($n = 80000$, SIX RANGE PARAMETERS): LARGEST LOG-LIKELIHOOD DEVIATION FROM NATIVE AND SPEED RELATIVE TO NATIVE (MEAN ± 95% t INTERVAL OVER 6 ROUNDS; TRUNCATION ROUNDS TO NEAREST). AT 1000 W TRUNCATION ALSO USES 3.9 ± 0.7% (36 BITS) AND 8.2 ± 0.6% (28 BITS) LESS ENERGY.

	Trunc. 36	Trunc. 28	cuBLAS emu. 36	FP32
max $\lvert\Delta\ell\rvert$	2.4×10^{-5}	7.2×10^{-4}	2.2×10^{-3}	87
1000 W	+3.0 ± 0.3%	+3.8 ± 0.4%	−41%	−8.2%
700 W	+5.0 ± 0.1%	+7.9 ± 0.2%	−31%	+3.2%
500 W	+4.8 ± 0.1%	+7.5 ± 0.1%	−21%	+3.9%

The log-likelihood is about $1.2\text{–}1.7\times10^5$, so a deviation of 7.2×10^{-4} at 28 bits is far below what matters for estimation (Table XI). At 1000 W, where this native factorization is capped 91–94% of the time, 28 bits is 3.8% faster (36 bits 3.0%), but the 1000 W gain depends on the setting: with the Morton ordering used below ($n = 60000$), where native is capped in only 19–54% of the samples, 28 bits gains +0.0/+0.1%, and the maximum-likelihood fit below gains 1.8–2.3%. FP32's deviation makes it unusable for estimation; it may stem from the conditioning of the covariance, which worsens with n (at $n = 8000$ FP32 still finds the same estimate). cuBLAS emulation is less accurate than 28-bit truncation and slower, partly because the trailing update's inner dimension equals the block size, 1024, a shape that already favors truncation in the crossover model (Section VII-B). The per-row span is wider than in the DFT case, yet emulation does not go wrong, only loses accuracy: a wide per-row span is necessary but not sufficient for emulation failure, which also depends on how much the dropped small elements contribute.

A one-parameter maximum-likelihood fit. To check that the estimate itself is unchanged, we fitted the range parameter with variance and smoothness fixed: golden-section search over its logarithm, 17 likelihood evaluations per variant ($n = 60000$; 1000 and 700 W; two interleaved rounds). Truncation to 36 and 28 bits and cuBLAS emulation return the native estimate $\hat{\theta} = 0.05007952$ in every printed digit, with maximized log-likelihoods within 4.8×10^{-6} (36 bits) and 2.3×10^{-4} (28 bits) of native; FP32 moves $\hat{\theta}$ by 0.49% and loses 0.72 in log-likelihood. At 700 W the fit is 7.1–7.3% faster with 28 bits and 4.8–4.9% faster with 36 bits, using 6.6–6.8% and 4.5–4.8% less energy; at 1000 W it is 1.8–2.3% and 1.6–2.2% faster with 9.1–9.9% and 3.5–5.6% less energy; cuBLAS emulation is 32–41% slower. Of the three whole-matrix choices we tested (FP32, 36-bit cuBLAS emulation and truncation), only truncation is both fast and accurate on this wide-range, power-bound workload.

Composing with tile mixed precision. The mixed-precision Cholesky factorizations of the ExaGeoStat line change the format of whole tiles instead (Section II). To see whether truncation still helps there, we ordered the locations along a Morton curve and added such a variant to our kernel: in the trailing update, tiles below each column's FP64 extent,

chosen by the tile-norm rule with a required accuracy of 10^{-9} as in [47], use FP16 inputs with FP32 accumulation on the tensor cores (52% of the tiles at range 0.02, 3% at 0.08). Over the six ranges (n = 60000, two ABBA rounds) it runs 1.60–1.63× faster than native FP64 at 1000, 700 and 500 W, with log-likelihood deviations up to 0.043. Truncating its remaining FP64 tiles to 28 bits makes it another 1.9% faster at 1000 W and 7.0% and 6.1% faster at 700 and 500 W, with an unchanged deviation, so truncation composes with format changes rather than competing with them. Zeroing the FP16 operands to 5 mantissa bits as well added nothing.

ExaGeoStat itself. We also built ExaGeoStatCPP (StarPU and Chameleon tile Cholesky with cuBLAS 13.6; tile size 1920) and ran its MLE (N = 60000, six BOBYQA likelihood evaluations per run, data generated once natively and read by every run) with the shim on its FP64 tile GEMMs. Matérn covariance generation runs on the CPU and takes about 9 s per evaluation against 2.3 s of GPU factorization at 1000 W, and the factorization is partly power-limited even there (GPU power about 620–670 W on average, frequent power-cap events). Relative to a baseline that copies the operands without truncating, the factorization is 3.0% (36 bits) and 4.0% (28 bits) faster at 1000 W, 5.0% and 8.0% at 600 W, and 5.6% and 9.4% at 450 W (means of two ABBA rounds; at 450 W the rounds differ widely, +6.9 and +11.9% with 28 bits at 707 and 926 MHz, because the CPU shares the module cap while generating the covariance); with 28 bits the mean SM clock during factorization rises from 1913 to 1975 MHz at 1000 W and from 1166 to 1305 MHz at 600 W. The log-likelihoods change by at most 1.2×10^{-10} and 9.8×10^{-8} relative. The shim's own copy costs 7–9%, which leaves it 0.5–3.1% slower than unmodified ExaGeoStat, so the gain needs an implementation inside the library. ExaGeoStatCPP's mixed-precision option (`--computation=mixed`) is only an enumeration value without an implementation, so we could not compare with its own mixed precision.

VIII. AI Workloads: Training and Inference

MLPerf Training measures time to target quality (eval loss ≤ 0.925 on GovReport for Llama 2 70B LoRA); a faster step that needs more samples to converge loses.

Training: truncating the FP8 base weights is a negative result. We truncated only the frozen FP8 base weights of Llama 2 70B LoRA (once, after loading); activations, gradients, LoRA weights and updates and per-tensor scaling follow [52]. On 8 nodes with the same seed, native FP8 at 575 W reached the target after 3456 samples in 28.34 min. Keeping 2 bits made each 384 samples only 0.9–1.2% faster but needed 4224 samples (22% slower at 575 W, 19% at 800 W); 1 bit did not converge (it also lowers the largest representable E4M3 value from 448 to 384, compressing the top weights by about 14%). The eval-loss curve shifts up by 0.006–0.008, while the train-loss gap varies (with 2 bits −0.031 at the first logged step, then +0.003 to +0.017): truncation weakens the base model at load time, and a rank-16 adapter does not compensate. A second seed (5678, 800 W) agrees: native 3072 samples in 22.74 min, 2 bits 4224 samples in 31.56 min (+39%). Across two seeds and four pairs (one at 575 W, three at 800 W), the first evaluation is always 0.008–0.010 higher and convergence takes 2–3 more evaluation intervals. Native convergence differs by one interval between node sets (3072 versus 3456), so we read this only as evidence that FP8's 3 mantissa bits leave no room. The rounding mode shows no consistent order (at 800 W truncation and stochastic rounding converge at 4224 samples, nearest at 5376). Truncating only MLP layers needs 4224 samples, attention only 3840 (one interval, within node-set noise, but attention is only about 18% of the linear weights); the first-evaluation gaps of +0.0064 (MLP) and +0.0028 (attention) roughly add up to the +0.0086 of full truncation (their sum is 0.0092). A repeat of the attention-only run gave nearly the same first evaluation (0.9428 versus 0.9425) but converged one interval later (4224 versus 3840, missing the target by 10^{-5} at 3840), so the first-evaluation gap, not the convergence point, is the robust metric.

TABLE XII
WikiText-2 perplexity (sequence 2048) relative to BF16 for the linear layers' formats; FP8 uses real FP8 tensor-core GEMMs, and kept bits are rounded to nearest (ties to even).

Format	L2-7B	L3.1-8B	Qw2.5-7B	Mis-7B	OPT-6.7B	L2-70B
BF16, 4 b	+0.09	+0.19	+0.13	+0.09	+0.45	+0.10
BF16, 3 b	+0.44	+1.02	+0.65	+0.41	+2.00	+0.51
FP8 per tensor	+0.63	+0.95	+0.70	+0.45	+2.23	+0.48
FP8 per channel	+0.32	+0.88	+0.61	+0.34	+0.69	+0.42
FP8 p.c., 2 b	+3.2	+6.5	+4.0	+2.5	+7.0	+3.4
FP8, wts. 2 b	+1.9	+4.0	+2.6	+1.3	+2.8	—

Values in percent. b: kept bits; L: Llama, Qw: Qwen, Mis: Mistral; p.c.: per channel; wts.: weights only.

Inference: FP8 has already taken most of the benefit. BF16 rounded to nearest at 3 bits has as many mantissa bits as FP8 E4M3 and matches its accuracy on six language models (Table XII; with truncation, 4 bits are needed for a similar GEMM error, 4.35% on uniform data). We then compared the two where FP8 is known to fail: attention perplexity at 2k/8k context (BF16 3 bits +0.49/+1.06%, emulated FP8 attention +0.52/+1.08%), key retrieval at 8k–32k context on Llama-3.1-8B (both 100%), BERT-large SQuAD (all formats above 99.9% of FP32 F1) and CogVideoX-2B video generation (PSNR against native 18–38 dB for FP8 attention, 20–29 dB for BF16 3 bits). With proper scaling the two tie, and FP8 GEMMs are about 1.7–1.9× as fast (at 575 W, 1.67–1.72× BF16 with 3 bits and 1.80–1.87× native BF16). A naive FP8 attention that does not scale the softmax probabilities fails as sequences grow (74% error at 4096 tokens), and on CogVideoX-2B (about 18k tokens) it changes the composition and adds noise (PSNR 10–12 dB, SSIM 0.07–0.66) where BF16 with 3 bits keeps 20–21 dB and SSIM 0.82–0.91. This is a range problem, not a mantissa problem: the smallest positive E4M3 value, about 2^{-9}, exceeds the mean attention probability over 18k tokens, while BF16 has 8 exponent bits; correct implementations already fix it inside FP8 [54].

Without switching to 2:4 structured sparsity or INT4

weights, zeroing on top of FP8 is the remaining way to go faster than FP8 on Hopper, which lacks FP4. On real layer-40 activations and weights of Llama2-70B (8192×57344×8192), keeping 2 bits on both sides is 4.4/3.2/5.0/8.2% faster at 1000/700/575/450 W, weights only 1.3–3.3%, but 70B perplexity rises by 3.4%. Per-layer sensitivity shows q/k/v as least sensitive but a small share of the compute (with grouped-query attention, q is about 7.8% and k and v about 1% each), and the MLP layers as most of the compute but more sensitive. FP8's advantage comes mainly from halving the data path: at 575 W BF16 with 3 bits is only 8% faster than BF16, while FP8 is 1.8×. Power savings are limited too. Decode (64 tokens) is bandwidth-bound; at 1000 W zeroing leaves its speed unchanged on uniform data, though on real Llama2-70B weights zeroed FP8 is still 2.6–3.0% faster at the same clock. FP8 zeroed to 0 bits saves 104 W on uniform data but only 61 W on real Llama2-70B weights, and truncating weights to 2 or 0 bits saves only 1.5% or 5.5% of power at an accuracy cost. Below 575 W these savings turn into 2–16% speedups, and prefill is power-bound even at 1000 W.

For weights and activations that directly set model quality and have no residual correction, zeroing turns directly into quality loss, in contrast with solvers such as HPL-MxP and SCF that correct errors. For GPUs without FP8 tensor cores, or pipelines that must stay in BF16, BF16 with 3–4 kept bits is a nearly lossless option; for FP8 deployments, zeroing saves only 1.5–5.5% of power in memory-bound phases, at an accuracy cost.

IX. Discussion

Benefit under a fixed power budget. Under a fixed power budget, such as a rack limit, the equivalent power saving becomes usable compute. At 8 nodes × 700 W, tuned HPL with the safe 36 bits rises from 346.2 to 362.9 TFLOP/s on set A (+4.8%; set B +3.9%); native HPL would need about 768 W per node (774 W on set B), about 0.5–0.6 kW more for the cluster, to match. At the default 1000 W cap the gain is mainly energy: 7.5–8.6% less per solve, with 0.6–3.9% more speed depending on how far the cap (and temperature) lowers the native clock. The cost is one bitwise operation per element, which can be fused into the kernel that produces the operand, and no hardware. At 575 W the SM clocks of our 14 nodes differ by 17% (1129–1325 MHz), and synchronous runs move at the pace of the slowest node, as earlier work found for CPUs under power bounds [18], [19]; zeroing and variation-aware per-node caps are complementary.

Benchmark rules. The TOP500 Linpack rules require full 64-bit floating-point arithmetic and explicitly exclude solving in lower precision with iterative refinement, and emulating floating point with integer arithmetic [63]; HPL on truncated operands does not meet the full-precision requirement either. Since June 2025 the HPL-MxP rules require at least 16-bit floating point and allow diagonally dominant test matrices for testing only [60]; BF16 keeping 3 bits is effectively a 12-bit format, and our test matrix is not allowed for official runs. All HPL and HPL-MxP numbers here are therefore research results, not official scores. MLPerf Inference closed division allows any mathematical quantization but forbids pruning [55]; zeroing is the former. Whether MLPerf Training permits truncated weights depends on each round's rules; in our results uniform truncation does not pay anyway. Scientific software has no such restriction; the criterion is whether physical results stay within the required accuracy.

A hardware counterexample. An Altera patent states that zeroing low bits at the input of an FPGA floating-point multiplier has "barely any effect on reducing the amount of power" because the compression tree keeps switching [64]. This does not contradict our measurements: the patent is a qualitative statement about a 27×27 fixed-point multiplier, whereas on a GPU the truncated bits also travel through registers, operand buses and memory, whose toggles disappear as well. Table I supports this: a pure memory copy saves 18% of module power without any multiplication, 97% of it from the absence of toggles, while in the GH200 multipliers zeros still save more than ones (Section III).

Limitations and threats to validity. (i) All experiments use one GH200 cluster, and GH200 favors truncation: its FP64 tensor cores are strong, so INT8 emulation gains least there (DGEMM emulation runs 18.5× faster than native on an RTX 5080 but 1.4× on A100 and GH200 [12]); where FP64 is weaker the crossover may reverse, and other generations manage power differently (an earlier report has preliminary H200 and MI350X results). (ii) The HPL residual is a deterministic but chaotic function of N (28 bits passes at $N = 131072$, 262144 and 286720, all close to the threshold, and fails at 196608), so operating points (36 bits for HPL, 3 for HPL-MxP) were validated at several sizes. (iii) HPL-MxP used a diagonally dominant test matrix (diagonal equal to the row's off-diagonal magnitude sum) with approximate diagonal factorization, which the rules allow only for testing [60]; its stability at 3 kept bits may be a property of this matrix. (iv) Each scientific code was tested on one or two input systems; forces, geometry optimization and larger systems are not covered. (v) The shim costs 3–9% for copies depending on the application (about 5% for Quantum ESPRESSO, 7–9% for ExaGeoStat), so shim results are always compared with a baseline taking the same copy path; in-program zeroing has no such cost. (vi) Nodes differ by 13–17% at equal power, so all comparisons are paired on the same nodes by ABBA order or by seed; intervals treat pairs within a job or node set as independent; moving HPL to a second node set (measured on a day with a slower network) lowered its gain at 600–800 W by 0.5–0.9 points, more than pairs within a job differ, so node sets are a larger source of uncertainty than run-to-run noise; and the crossover sweep measured its four values of n on different nodes. (vii) The default 1000 W cap is also the highest, not an uncapped state; the limiter's time scale is unknown; and caps below 450 W lie outside the datasheet range [56]. (viii) Cooling and the chassis supply change conclusions: at low fan speed the GPU hits its thermal limit first, and at full fan speed native HPL-MxP at 1000 W trips the hardware power brake; these conditions must be reported. (ix) The cluster is

shared and another user's tool reset caps at least twice, so every window checks the cap in effect. (x) Power comes from the driver's sensor, not an external meter, and on A100/H100 it samples only part of the time [36]; our comparisons are paired relative values on one measurement path with steady windows of 10–20 s, but absolute energy should be cross-checked with an external meter; module power also includes the Grace CPU and memory (the CPU competes for the same cap when ExaGeoStat generates the covariance), so all energy figures are relative values on one path. (xi) The fill-pattern controls cover only 0 and 20 kept bits, the 500 W rows come from one node and two runs, and the extra saving of zeros at 0 kept bits is unexplained. (xii) Not all of the Quantum ESPRESSO gain is a power effect: on the gold surface about half comes from fewer Davidson iterations, for reasons not yet identified. (xiii) The real-data emulation comparisons on Quantum ESPRESSO ran once per setting, and GEMMul8 ran out of GPU memory on two applications. (xiv) The Gaussian-process results come from our ExaGeoStat-like kernel on synthetic data, and its mixed-precision variant is a simplified re-implementation (FP64 storage, one FP16 GEMM per panel over the tiles that need it), not the ExaGeoStat code; ExaGeoStat itself was measured only through the shim, whose copy cost offsets the gain.

X. Conclusion

On a production GH200 cluster, truncating unused mantissa bits in software buys real performance for real HPC applications under power caps while their physical results stay within each code's own criterion. When the GPU is power-bound and truncation is done inside the program, the HPC programs we tested that check their own results all gain: at 500–800 W, tuned HPL keeping 36 significand bits is 3.3–4.7% faster on two node sets (42–74 W equivalent at 500–700 W) and HPL-MxP keeping 3 bits 1.7–5.8% on a diagonally dominant test matrix; Quantum ESPRESSO with tolerance-driven bit counts is 2.1–3.6% faster at 450–550 W with every printed digit of the energies unchanged (on the gold surface about half of the gain comes from fewer Davidson iterations); power-hungry DF-MP2 keeping 20–32 bits is 4.7–7.7% faster at 500–700 W; and wide-range, power-bound Gaussian-process geostatistics keeping 28 bits is 7.5–7.9% faster at 500–700 W. At the default 1000 W cap HPL uses 7.5–8.6% less energy per solve and is 0.6–3.9% faster, depending on how far the cap (and temperature) lowers the native clock. Microbenchmarks give upper bounds (FP64 GEMM keeping 27 bits: 6.6–21% at 300–1000 W; pure data movement: 18% less module power at 1000 W), not application operating points. The gain comes mainly from fewer toggles in arithmetic units and data paths, so it suits FP64 solvers with residual correction or self-consistent iteration whose inputs flow directly and in volume into multipliers. Its scope is also clear: the crossover with INT8 emulation of FP64 moves with matrix shape and cap, and a three-parameter model in (m, n, k) describes our measurements but still needs tests near the crossover; on data with a wide per-row exponent span, fixed-width row-sliced emulation can silently go wrong, while more slices or cuBLAS's automatic mode are correct but slower than truncation; replacing every GEMM with FP32 or 3xTF32 fails on long inner products; kernels whose intermediates regrow full mantissas gain little; and for FP8 AI inference, further zeroing trades accuracy for speed. Mantissa zeroing costs one bitwise operation per element, which can be fused into the kernel that produces the operand, needs no hardware support, and complements format changes, including tile-level mixed precision. All HPL, HPL-MxP and MLPerf numbers here are research results, not official scores.

Data and Code Availability

The mantissa shim, the in-program truncation of each application, the Slurm scripts, the raw measurement logs, the summarized CSV files and the scripts that draw every figure are in the project repository (branch `gh200-mantissa-study`). The script `gh200/analysis/paper_figures.py` regenerates every figure of this paper from the CSV files in `gh200/results/`, including Fig. 5, whose points and their sources are listed in `gh200/results/range_map_points.csv`.

Appendix A
Artifact Description

Hardware and power control. Fourteen nodes, each with one NVIDIA GH200 480GB superchip (aarch64), 200 Gb/s InfiniBand, Slurm partition `compute`. Module power caps are set with the cluster's `nv_power` tool only on exclusively allocated nodes; every job restores 1000 W on exit and records the cap in effect in each measurement window. Fans were left at the operator's setting.

Software. NVHPC 26.5 (CUDA 13.2, cuBLAS 13.4, NCCL 2.29, HPC-X Open MPI 5), driver 615.71; ExaGeoStatCPP links cuBLAS 13.6. HPL is a CUDA port of rocHPL (branch `feat/cuda-port`) and HPL-MxP the `perf/h200-mantissa` branch of an HPL-MxP implementation [33]; Quantum ESPRESSO 7.6 [57]; GPU4PySCF 1.8.1 [58], [59]; MFEM 4.10.1 (CUDA, partial assembly); SW4 (`raja` branch, commit 52b51c9, RAJA 2026.07.1); LAMMPS 22 Jul 2025 Update 6 (KOKKOS, CUDA); GEMMul8 3.5.1; MLPerf Training Llama 2 70B LoRA with NVIDIA's v5.0 implementation. Analysis uses Python with pandas, NumPy and Matplotlib.

Code and data. Table XIII lists where each part lives; the experiment logs in `gh200/` record every job, including failed and repeated ones, and every table and figure names its Slurm jobs and result files there.

Reproduction. Each experiment is one Slurm job that sets caps only on its own nodes, waits for the GPU to cool to 45 °C before each run, interleaves variants in ABBA order and writes nvidia-smi logs next to the program output; an analysis script then joins run windows with the power logs. For example, the crossover sweep of Section VII-B is `slurm/e1_gemm_tdp.sbatch` with the shapes, caps and methods passed as environment variables, summarized

TABLE XIII
WHERE EACH MECHANISM AND EXPERIMENT LIVES IN THE REPOSITORY.

Path	Content
`gh200/bench/`	cuBLAS mantissa GEMM microbenchmark (Sections VI-A, VII-B)
`gh200/shim/mantshim.cu`	cuBLAS interposer: rounding modes, cuBLAS emulation (fixed and automatic, bit-count log), FP32/3xTF32 replacement, exponent-span statistics
`rocHPL/src/blas/HPL_quantize_device.cpp`	HPL operand rounding (`HPL_QUANT_*_BITS`)
`rocHPL-MxP/src_mi350/hplmxp.h`	HPL-MxP zeroing on BF16 write-back (compile-time `MANT_BITS`)
`apps/q-e/KS_Solvers/Davidson/`	in-program truncation and the tolerance-driven rule (`QE_MANT_ADAPT`)
`apps/pyscf-inputs/mp2_variants.py`	DF-MP2 variants: in-place truncation, FP32 path, cuBLAS emulation
`apps/geostat/gp_mle.py`	Gaussian-process likelihood and MLE: truncation, emulation, FP32, Morton order, tile mixed precision
`apps/patches/`	MFEM, SW4, LAMMPS and ExaGeoStatCPP changes
`gh200/slurm/`	job scripts (cap sweeps with ABBA order, HPL tuning, shim runs)
`gh200/analysis/`	summaries, ΔW, intervals, crossover fit, figures
`gh200/results/`	summarized CSV files behind every table and figure

by `analysis/crossover.py`; the Quantum ESPRESSO runs of Section VI-D are `slurm/sci_sweep.sbatch` with `analysis/sci_summary.py` and `analysis/qe_adapt_ci.py`; and the cap sweeps of Section VI-B are `slurm/hpl_tdp.sbatch` (`BENCH=hpl` or `mxp`) with `analysis/summarize.py`.

REFERENCES

[1] S. Bhalachandra, B. Austin, S. Williams, and N. J. Wright, "Understanding the impact of input entropy on FPU, CPU, and GPU power," arXiv preprint arXiv:2212.08805, 2022.

[2] T. Gregersen, P. Patel, and E. Choukse, "Input-dependent power usage in GPUs," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1872–1877.

[3] H. He, "Strangely, matrix multiplications on GPUs run faster when given "predictable" data! [short]," Thonk From First Principles (Substack blog), https://www.thonking.ai/p/strangely-matrix-multiplications, Apr. 2024.

[4] O. Antepara *et al.*, "Benchmark-driven models for energy analysis and attribution of GPU-accelerated supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, 2025, pp. 888–904.

[5] NVIDIA, "Performance benchmarking — NVIDIA TensorRT (GPU power consumption and power throttling)," https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/benchmarking.html, accessed Sep. 30, 2026.

[6] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, "Reducing power by optimizing the necessary precision/range of floating-point arithmetic," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 8, no. 3, pp. 273–286, 2000.

[7] J. Pool, A. Lastra, and M. Singh, "Energy-precision tradeoffs in mobile graphics processing units," in *IEEE International Conference on Computer Design (ICCD)*, 2008, pp. 60–67.

[8] ——, "Precision selection for energy-efficient pixel shaders," in *ACM SIGGRAPH Symposium on High Performance Graphics (HPG)*, 2011, pp. 159–168.

[9] D. Guenther, T. Henriksson, R. Leupers, and G. Ascheid, "Mantissa-masking for energy-efficient floating-point LTE uplink MIMO baseband processing," in *Design, Automation & Test in Europe (DATE)*, 2016, pp. 1028–1029.

[10] P. V. Argade and D. N. Peroni, "Variable precision computing system," US Patent 11,455,766 B2, Sep. 27, 2022.

[11] K. Ozaki, Y. Uchino, and T. Imamura, "Ozaki scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique," arXiv preprint arXiv:2504.08009, 2025.

[12] Y. Uchino *et al.*, "High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines," in *SC '25 Workshops*, 2025.

[13] N. Chatterjee *et al.*, "Architecting an energy-efficient DRAM system for GPUs," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 73–84.

[14] S. Ghose *et al.*, "What your DRAM power models are not telling you: Lessons from a detailed experimental study," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, 2018.

[15] O. Antepara, L. Oliker, and S. Williams, "Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200," in *SC '25 Workshops*, 2025, pp. 1205–1216.

[16] A. Ujeniya, J. Eitzinger, G. Hager, and G. Wellein, "Architectural tradeoffs in the energy-efficient era: A comparative study of power-capping NVIDIA H100 and H200," arXiv preprint arXiv:2604.11391, 2026.

[17] J. Lucas and B. Juurlink, "ALUPower: Data dependent power consumption in GPUs," in *IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2016, pp. 95–104.

[18] B. Rountree, D. H. Ahn, B. R. de Supinski, D. K. Lowenthal, and M. Schulz, "Beyond DVFS: A first look at performance under a hardware-enforced power bound," in *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW)*. IEEE, 2012, pp. 947–953.

[19] Y. Inadomi *et al.*, "Analyzing and mitigating the impact of manufacturing variability in power-constrained supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 1–12, article 78.

[20] T. Patki, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, "Exploring hardware overprovisioning in power-constrained, high performance computing," in *Proceedings of the 27th International ACM Conference on Supercomputing (ICS '13)*. ACM, 2013, pp. 173–182.

[21] J. You, J.-W. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing GPU energy consumption of DNN training," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 119–139. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/you

[22] A. Angerd, E. Sintorn, and P. Stenström, "A framework for automated and controlled floating-point accuracy reduction in graphics applications on GPUs," *ACM Transactions on Architecture and Code Optimization*, vol. 14, no. 4, 2017, article 46.

[23] R. Jahadi and E. Atoofian, "Low-power register file for tensor cores," in *IEEE 15th International Green and Sustainable Computing Conference (IGSC)*, 2024, pp. 159–164.

[24] A. Dawson and P. D. Düben, "rpe v5: An emulator for reduced floating-point precision in large numerical simulations," *Geoscientific Model Development*, vol. 10, pp. 2221–2230, 2017.

[25] N. J. Higham and S. Pranesh, "Simulating low precision floating-point arithmetic," *SIAM Journal on Scientific Computing*, vol. 41, no. 5, pp. C585–C602, 2019.

[26] M. O'Connor *et al.*, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[27] G. Pekhimenko, E. Bolotin, N. Vijaykumar, O. Mutlu, T. C. Mowry, and S. W. Keckler, "A case for toggle-aware compression for GPU systems,"

in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016, pp. 188–200.

[28] C. Rubio-González *et al.*, "Precimonious: Tuning assistant for floating-point precision," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.

[29] N. J. Higham and T. Mary, "Mixed precision algorithms in numerical linear algebra," *Acta Numerica*, vol. 31, pp. 347–414, 2022.

[30] A. C. I. Malossi *et al.*, "The transprecision computing paradigm: Concept, design, and applications," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1105–1110.

[31] T. Grützmacher, T. Cojean, G. Flegar, H. Anzt, and E. S. Quintana-Ortí, "Acceleration of PageRank with customized precision based on mantissa segmentation," *ACM Transactions on Parallel Computing*, vol. 7, no. 1, pp. 1–19, 2020.

[32] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham, "Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC18)*, 2018, pp. 603–613.

[33] J. Dongarra and P. Luszczek, "HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation," *The International Journal of High Performance Computing Applications*, vol. 40, no. 1, pp. 52–62, 2026, published online Sep. 25, 2025; arXiv:2509.19618.

[34] M. Klöwer, M. Razinger, J. J. Dominguez, P. D. Düben, and T. N. Palmer, "Compressing atmospheric data into its real information content," *Nature Computational Science*, vol. 1, no. 11, pp. 713–724, 2021.

[35] M. Patrou *et al.*, "Power-capping metric evaluation for improving energy efficiency in HPC applications," in *ISC High Performance 2025 Workshops (EESP)*, ser. LNCS, 2025, pp. 231–244.

[36] Z. Yang, K. Adamek, and W. Armour, "Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU's built-in power sensor," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC24)*, 2024.

[37] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications," *Numerical Algorithms*, vol. 59, no. 1, pp. 95–118, 2012.

[38] H. Ootomo, K. Ozaki, and R. Yokota, "DGEMM on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.

[39] NVIDIA, "cuBLAS library documentation, section 1.5: Floating point emulation (v13.4; archived 13.2.1)," https://docs.nvidia.com/cuda/cublas/index.html#floating-point-emulation, accessed Oct. 5, 2026.

[40] A. Schwarz *et al.*, "Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme," in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia)*, 2026, pp. 91–101.

[41] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, and D. E. Keyes, "ExaGeoStat: A high performance unified software for geostatistics on manycore systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 12, pp. 2771–2784, 2018.

[42] ——, "Geostatistical modeling and prediction using mixed precision tile Cholesky factorization," in *IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2019, pp. 152–162.

[43] S. Abdulah *et al.*, "Accelerating geostatistical modeling and prediction with mixed-precision computations: A high-productivity approach with PaRSEC," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 964–976, 2022.

[44] Q. Cao *et al.*, "Reshaping geostatistical modeling and prediction for extreme-scale environmental applications," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–12.

[45] J. Ren, H. Ltaief, S. Abdulah, and D. E. Keyes, "Accelerating mixed-precision out-of-core Cholesky factorization with static task scheduling," in *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, 2025, pp. 1–12.

[46] S. Abdulah *et al.*, "Boosting earth system model outputs and saving PetaBytes in their storage using exascale climate emulators," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1–12.

[47] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, and G. Bosilca, "Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion," in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 330–342.

[48] R. Olivares-Amaya, M. A. Watson, R. G. Edgar, L. Vogt, Y. Shao, and A. Aspuru-Guzik, "Accelerating correlated quantum chemistry calculations using graphical processing units and a mixed precision matrix multiplication library," *Journal of Chemical Theory and Computation*, vol. 6, no. 1, pp. 135–144, 2010.

[49] J. Woo, S. Kim, and W. Y. Kim, "Dynamic precision approach for accelerating large-scale eigenvalue solvers in electronic structure calculations on graphics processing units," *Journal of Chemical Theory and Computation*, vol. 19, no. 5, pp. 1457–1465, 2023.

[50] H. Huang, W. Shao, and J. Hammond, "Accelerating density fitting with adaptive-precision and 8-bit integer on AI accelerators," arXiv preprint arXiv:2601.08077, 2026.

[51] W. Dawson, K. Ozaki, J. Domke, and T. Nakajima, "Reducing numerical precision requirements in quantum chemistry calculations," *Journal of Chemical Theory and Computation*, vol. 20, no. 24, pp. 10 826–10 837, 2024.

[52] P. Micikevicius, D. Stosic *et al.*, "FP8 formats for deep learning," arXiv preprint arXiv:2209.05433, 2022.

[53] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 37, 2015, pp. 1737–1746.

[54] J. Zhang, H. Huang, P. Zhang, J. Wei, J. Zhu, and J. Chen, "SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, arXiv:2411.10958.

[55] MLCommons, "MLPerf inference rules, section 8.1: Weight definition and quantization," https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc, accessed Oct. 5, 2026.

[56] NVIDIA, "NVIDIA GH200 Grace Hopper superchip datasheet," Document 3773000, linked from https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/, Mar. 2025, accessed Oct. 5, 2026.

[57] P. Giannozzi *et al.*, "Quantum ESPRESSO toward the exascale," *The Journal of Chemical Physics*, vol. 152, p. 154105, 2020.

[58] X. Wu *et al.*, "Enhancing GPU-acceleration in the Python-based simulations of chemistry frameworks," *WIREs Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.

[59] R. Li, Q. Sun, X. Zhang, and G. K.-L. Chan, "Introducing GPU acceleration into the Python-based simulations of chemistry framework," *The Journal of Physical Chemistry A*, vol. 129, no. 5, pp. 1459–1468, 2025.

[60] HPL-MxP, "HPL-MxP rules," https://hpl-mxp.org/rules.md, accessed Oct. 5, 2026.

[61] M. Fasi, N. J. Higham, M. Mikaitis, and S. Pranesh, "Numerical behavior of NVIDIA tensor cores," *PeerJ Computer Science*, vol. 7, p. e330, 2021.

[62] F. A. Khattak and M. Mikaitis, "Accurate models of NVIDIA tensor cores," arXiv preprint arXiv:2512.07004, 2025.

[63] TOP500.org, "The Linpack benchmark," https://www.top500.org/project/linpack/, accessed Oct. 5, 2026.

[64] M. Langhammer, "Low power optimizations for a floating point multiplier," US Patent 9,606,608 B1 (Altera Corporation), 2017.